DATE DE DÉPÔT : 12/06/2026

Université de Lille – BUT SD – Semestre 6 - 2025/2026

RAPPORT DE SAE 6 : Outil décisionnel, Modélisation statistique et Intelligence Artificielle

Partie 1 : Intégration et visualisation de données

Présentées et soutenues par

Hoang Ngoc Lam LE

Minh Anh NGUYEN

Le 19 juin 2026

Sous la direction de

M. Yannick LE VELER

Remerciements

La réussite de mon projet n'aurait été possible sans l’appui éclairé, actif et judicieux de certaines personnes que nous souhaitons sincèrement remercier.

Nous tenons tout d’abord à remercier notre enseignant et encadrant de projet, M. Yannick LE VELER, pour son suivi tout au long de cette SAE. Ses connaissances en modélisation de bases de données, en conception décisionnelle ainsi qu’en intégration de données nous ont permis de mieux comprendre les fondamentaux des modèles Entité/Association et des architectures décisionnelles. Ses conseils méthodologiques, ses explications ainsi que les exercices pratiques proposés ont fortement contribué à l’avancement de notre projet.

Nous remercions également Mme Fatma BOUALI et M. Le Goff pour les enseignements apportés en bases de données relationnelles, en gestion de projet et en systèmes décisionnels. Les compétences acquises au cours de leur enseignement nous ont été particulièrement utiles pour aborder les différentes étapes liées à la modélisation, à l’intégration et à la visualisation des données.

Enfin, nous remercions l’ensemble de l’équipe pédagogique ainsi que les intervenants professionnels du BUT Science des Données de l’IUT de Lille - site de Roubaix, pour la qualité de la formation et l’accompagnement proposé tout au long de notre parcours universitaire.

Table des matières

INTRODUCTION **3**
MÉTHODOLOGIE **5**
1. Organisation du travail 5
2. Analyse initiale et modélisation 5
3. Construction des couches de données 5
4. Création des Flux de Données 6
5. Construction du Tableau de Bord Power BI 6
6. Validation et Optimisation 7
7. Perspectives 7
DÉTAILS DU PROJET **8**
1. Cadre général du projet 8
1.1. Contexte & Objectif du projet 8
1.2. Gestion du temps 8
1.3. Organisation du travail 8
1.4. Matériel, outils et technologies utilisés 9
2. Analyse initiale et modélisation 9
2.1. Origine des données (notaires, DGFiP) et les logiciels métiers utilisés 9
2.2. Dictionnaire 10
2.3. Analyse du fichier source 12
2.4. MCD, MLD et MPD 15
2.5. Scripts SQL 16
3. Intégration des Données 17
3.1. Structure DSA, ODS, DWH 17
3.2. Création des flux de données 20
3.3. Validation et Optimisation 30
4. Construction du Tableau de Bord Power BI 31
4.1. Définition des indicateurs de pilotage 31
4.2. Extraction des données du Data Warehouse 31
4.3. Modélisation des données dans Power BI 33
4.4. Création des mesures DAX 34
4.5. Présentation des tableaux de bord 35
5. Perspectives 40
CONCLUSION **42**
1. Bilan de projet 42
2. Bilan personnel 43
GLOSSAIRE **44**
ANNEXE **45**

INTRODUCTION

Avec le développement des technologies numériques et l'augmentation importante des volumes de données disponibles, les systèmes décisionnels occupent aujourd'hui une place essentielle dans l'analyse et l'aide à la prise de décision. Dans ce contexte, les données foncières représentent une source d'information particulièrement intéressante pour étudier l'évolution du marché immobilier, analyser les dynamiques territoriales et produire des indicateurs utiles à l'analyse économique et géographique.

Le projet présenté dans ce rapport s'appuie sur les données DVF (Demandes de Valeurs Foncières), mises à disposition en Open Data sur la plateforme data.gouv.fr. Ces données, produites par la Direction Générale des Finances Publiques (DGFiP) à partir des actes notariés et des données cadastrales, recensent les transactions immobilières réalisées en France au cours des dernières années.

Le jeu de données exploité représente environ 20 millions d'enregistrements répartis sur une quarantaine de variables décrivant les mutations foncières, les parcelles cadastrales, les caractéristiques des biens immobiliers ainsi que leur localisation géographique. Cette volumétrie importante constitue un cas d'étude pertinent pour la mise en œuvre d'une architecture décisionnelle et de processus ETL adaptés au traitement de données massives.

L'objectif principal de cette SAE est de concevoir une chaîne décisionnelle complète permettant de transformer des données brutes en informations exploitables, puis de les intégrer dans un entrepôt de données afin de produire des analyses décisionnelles. Ce projet mobilise plusieurs compétences acquises au cours du BUT Science des Données, notamment en modélisation de bases de données, intégration de données, développement de processus ETL et création de tableaux de bord interactifs. Cette réalisation a été effectuée en binôme dans le cadre de cette SAE.

L'étude porte sur l'ensemble du territoire français. Les données analysées couvrent plusieurs départements répartis sur l'ensemble de la France, permettant d'obtenir une vision représentative du marché immobilier national et de réaliser des analyses comparatives entre les territoires. Une attention particulière a été portée à la diversité géographique des observations afin de permettre des analyses comparatives selon les territoires et les caractéristiques locales du marché immobilier.

Pour atteindre ces objectifs, le projet s'est déroulé en trois étapes principales :

- Analyser les données DVF afin de comprendre leur structure et concevoir les modèles de données nécessaires
- Développer un pipeline ETL en Python pour nettoyer, transformer et charger les données dans un entrepôt de données
- Concevoir des tableaux de bord interactifs sous Power BI afin de faciliter l'analyse des transactions immobilières selon différentes dimensions telles que l'année, la localisation ou le type de bien.

Ce rapport présente l'ensemble des travaux réalisés dans le cadre de cette SAE, depuis l'étude des données sources jusqu'à la construction du système décisionnel et à la réalisation des tableaux de bord interactifs.

MÉTHODOLOGIE

Le projet a été organisé en plusieurs étapes complémentaires, allant de l'analyse initiale des données et de leur intégration dans un entrepôt de données, jusqu'à la création de tableaux de bord interactifs. Cette méthodologie a permis de structurer le travail de manière cohérente, d'assurer la qualité des traitements réalisés et de faciliter le suivi global du projet.

1. Organisation du travail

La première étape du projet a consisté à organiser le travail avec un diagramme de Gantt afin de planifier les différentes étapes de la SAE, suivre l'avancement des tâches et mieux répartir le temps de travail entre les membres du groupe.

La répartition des tâches a été définie dès le début pour optimiser l'avancement du projet et favoriser la collaboration en s'appuyant sur les points forts des membres du groupe. Des échanges fréquents ont également été réalisés afin de vérifier la cohérence des traitements effectués, corriger certaines difficultés techniques et assurer le bon déroulement du projet.

2. Analyse initiale et modélisation

La phase d'analyse du jeu de données DVF permet de comprendre la structure et l'organisation des informations disponibles. Nous avons ainsi étudié en détail les colonnes, les types de données, les valeurs manquantes et les éventuelles incohérences afin d'identifier les relations entre les entités clés.

Cette étude facilite la création d'un dictionnaire de données et des modèles pour structurer et comprendre la base. Voici la signification complète de chaque terme que nous avons utilisé :

- **MCD** : Modèle Conceptuel de Données
- **MLD** : Modèle Logique de Données
- **MPD** : Modèle Physique de Données

3. Construction des couches de données

Après la phase de modélisation, nous avons mis en place une architecture en trois couches pour organiser et traiter efficacement les données. Cette structure facilite la maintenance, la réutilisation et l'évolution future du système décisionnel :

- **DSA (Data Staging Area)** : Cette couche intermédiaire reçoit les données extraites de la base SOURCE avant leur transformation et leur historisation.
- **ODS (Operational Data Store)** : Cette couche permet de stocker les données historiques et de suivre les modifications et ajouts au fil du temps.
- **DWH (Data Warehouse)** : Nous avons utilisé un modèle décisionnel en étoile (star schema), où les tables de faits sont au centre, reliées aux tables de dimensions, pour faciliter l'analyse des données.

4. Création des Flux de Données

La création des flux de données et du pipeline ETL représente une étape importante de cette SAE. Nous avons mis en place des flux de données pour intégrer et acheminer les informations à travers les différentes couches de la solution:

- **Des fichiers CSV (DVF) à la base Source**: Cette étape consiste à importer les données brutes issues des fichiers DVF dans une base relationnelle structurée selon le modèle Entité-Association. Les informations sont réparties dans différentes tables (mutation, parcelle, lot, adresse, commune, etc.) afin de faciliter leur stockage et leur exploitation.
- **De Source à DSA**: Avant chaque chargement, les tables de la couche DSA sont entièrement vidées (truncate) afin de garantir la cohérence des données transférées depuis la couche SOURCE et d'éviter la création de doublon. Cette approche permet d'éviter l'accumulation de doublons et de maintenir une cohérence entre les différentes couches.
- **De DSA à ODS**: Les données sont ensuite transférées vers l'ODS en ajoutant des informations d'historisation, notamment une date de chargement (load_date). Cette étape permet de conserver les différentes versions des données et d'assurer leur traçabilité dans le temps.
- **De ODS à Datawarehouse**: Les données sont intégrées dans le modèle décisionnel en étoile. Les dimensions sont historisées selon le principe du Slowly Changing Dimension (SCD Type 2) grâce à l'utilisation des colonnes DateDébut, DateFin et CurrentFlag. Seules les nouvelles données ou les modifications détectées sont intégrées afin de préserver l'historique tout en identifiant les enregistrements actuellement actifs.

5. Construction du Tableau de Bord Power BI

À partir des données du Data Warehouse, nous avons développé plusieurs tableaux de bord interactifs sous Power BI afin de faciliter l'analyse des données immobilières.

Les différentes pages permettent de :

- **Page Synthèse** : visualiser les principaux indicateurs du marché immobilier.
- **Page Localisation**: analyser la répartition des transactions selon les départements et les communes.
- **Page Mutations** : étudier le nombre et l'évolution des ventes immobilières.
- **Page Parcelles** : explorer les caractéristiques des parcelles cadastrales.
- **Page Types de Biens** : comparer les différents types de biens immobiliers et leurs caractéristiques.

Grâce aux filtres interactifs, les utilisateurs peuvent facilement explorer les données selon l'année, la localisation ou le type de bien.

L'objectif principal de ces tableaux de bord est de rendre les données plus lisibles et plus accessibles afin de faciliter l'analyse décisionnelle et l'identification des tendances du marché immobilier.

6. Validation et Optimisation

Plusieurs phases de validation et d'optimisation ont été réalisées pour garantir la fiabilité des données et le bon fonctionnement du pipeline ETL:

- Contrôle de l'intégrité: Des vérifications ont été faites après chaque chargement (nombre de lignes, cohérence des colonnes) pour éviter les doublons et les erreurs de clés (primaires/étrangères) dans MySQL Workbench.
- Validation des flux: Des tests sur les chargements incrémentaux ont permis de vérifier que seules les nouvelles données étaient ajoutées dans l'ODS et le DWH.
- Contrôle des données chargées: vérification du nombre d'enregistrements dans chaque couche (SOURCE, DSA, ODS et DWH).
- Contrôle de cohérence: comparaison de certains indicateurs calculés dans Power BI avec les résultats obtenus directement par requêtes SQL.

7. Perspectives

Plusieurs perspectives d'amélioration peuvent être envisagées pour enrichir ce projet:

- **Intégration de données complémentaires** : enrichir les analyses en ajoutant des données socio-économiques, démographiques ou géographiques provenant d'autres sources ouvertes.
- **Enrichissement des tableaux de bord** : Ajouter des fonctionnalités avancées dans Power BI (analyses statistiques poussées, cartographies détaillées et nouveaux indicateurs).

DÉTAILS DU PROJET

1. Cadre général du projet

1.1. Contexte & Objectif du projet

Dans ce projet, nous avons travaillé en binôme dans le cadre d'une SAE du BUT Science des Données, représentant environ 80 à 90 heures de travail réparties de mars à juin 2026.

L'enjeu est de concevoir une chaîne décisionnelle complète, allant du traitement des données brutes (DVF) jusqu'à leur intégration dans un entrepôt de données. Cette solution permet de centraliser et de structurer les informations afin de produire des tableaux de bord interactifs fiables, essentiels à l'analyse et à la visualisation décisionnelle.

1.2. Gestion du temps

Le travail de cette SAE a été organisé selon un calendrier prévisionnel, présenté sous forme de diagramme de Gantt, afin de suivre l'avancement de chaque phase du projet (Figure 1).

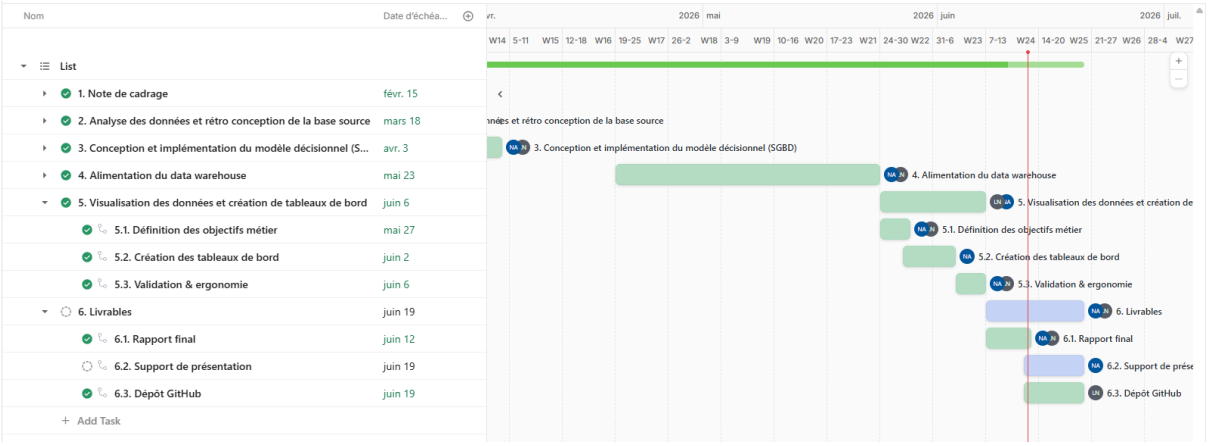


Figure 1. Structure de la fonction principale (main) pour l'orchestration du pipeline

1.3. Organisation du travail

Afin de rendre notre travail plus efficace, nous nous sommes réparties les tâches en fonction de nos compétences respectives. Nous avons choisi de travailler sur deux ordinateurs différents afin de garantir la synchronisation des données et d'éviter d'éventuels problèmes techniques. Cela nous a permis d'être réactives et de maintenir une bonne stabilité du projet.

La répartition des rôles était la suivante :

- Travail commun : Analyse initiale, création du dictionnaire de données, gestion de temps, rédaction du rapport.
- Membre 1 : Modélisation des base de données, nettoyage des données, création du Data Warehouse, intégration des données entre les différentes couches.
- Membre 2 : vérification des flux de données, recueil des besoins métiers, construction des tableaux de bord et préparation du support de présentation.

1.4. Matériel, outils et technologies utilisés

Les outils ont été sélectionnés en fonction de leur compatibilité technologique, de leur facilité d'intégration dans le pipeline et de leur performance pour la visualisation des données:

- **ClickUp**: outil de gestion de projet utilisé pour organiser les tâches et suivre l'avancement du projet.
- **MySQL Workbench**: Système de gestion de base de données relationnelle utilisé pour stocker la base source et exécuter les requêtes SQL.
- **Jupyter Notebook**: Environnement d'exécution et de documentation utilisé pour développer les scripts Python permettant d'exporter et de traiter les données.
- **Langages**: Python (pour les scripts ETL) et SQL (pour la manipulation des données).
- **Power BI**: Outil de Business Intelligence utilisé pour concevoir les tableaux de bord interactifs et de visualiser les indicateurs décisionnels.
- **Looping**: Nous avons utilisé cet outil pour gérer la création des schémas et la transformation des données dans le Data Warehouse.
- **Bibliothèques Python**: pandas, numpy, SQLAlchemy, mysql-connector-python, datetime et matplotlib.

2. Analyse initiale et modélisation

Le jeu de données DVF (Demandes de valeurs foncières) recense les transactions immobilières réalisées en France au cours des cinq dernières années. Il est produit par la DGFiP à partir des actes notariés et des données cadastrales, et constitue une source de référence pour l'analyse du marché immobilier.

2.1. Origine des données (notaires, DGFiP) et les logiciels métiers utilisés

Les données DVF (Demandes de Valeurs Foncières) proviennent du traitement informatisé mis en œuvre par la Direction Générale des Finances Publiques (DGFiP) et alimenté par la Base Nationale des Données Patrimoniales (BNDP). Elles sont issues des actes transmis aux services de la publicité foncière et de l'enregistrement lors des transactions immobilières réalisées sur le territoire français.

Le processus de production des données débute par la saisie des actes au sein des logiciels métiers utilisés par les notaires. Les informations sont ensuite intégrées dans les systèmes d'information de la DGFiP, où elles font l'objet de contrôles, de normalisations et d'enrichissements à partir de données cadastrales. Cette étape permet notamment d'ajouter des informations relatives aux parcelles, aux caractéristiques des biens immobiliers ainsi qu'à leur localisation géographique. Les identifiants cadastraux utilisés sont compatibles avec les référentiels ouverts diffusés par Etalab, facilitant ainsi leur exploitation dans des projets de valorisation de données.

La production de ces données mobilise différents outils spécialisés, notamment les logiciels métiers notariaux pour la saisie des actes, les systèmes d'information de la DGFiP pour l'agrégation et le contrôle des informations, ainsi que des outils de géocodage permettant d'associer les parcelles cadastrales à des coordonnées géographiques selon le système WGS-84.

L'analyse de l'origine et de la structure des données DVF a permis de mieux comprendre leur organisation, leurs contraintes ainsi que certaines limites techniques, telles que l'absence de géolocalisation pour certaines parcelles ou les modifications liées aux remaniements cadastraux. Cette étude constitue une étape essentielle pour réaliser la rétroconception de la base de données source et construire les modèles de données nécessaires à la suite du projet, notamment le Modèle Conceptuel de Données (MCD) et le Modèle Logique de Données (MLD) selon la méthode MERISE.

2.2. Dictionnaire

Nous avons créé un dictionnaire (Tableau 2) pour décrire chaque champ de chaque table, en précisant le nom, la description, la nature, le type, ainsi que les règles de calcul et d'intégrité associées à chaque champ. Cela nous a beaucoup aidées à mieux comprendre les données lors de leur transformation.

Table	Désignation	Description	Type	Clé	Règle d'intégrité
Mutation	id_mutation	Identifiant de la mutation	text	PK	non nul
Mutation	nature_mutation	Nature de la mutation	text	Aucune	non nul
Mutation	date_mutation	Date de la mutation	date	Aucune	non nul
Mutation	valeur_fonciere	Valeur foncière de la mutation	décimal / réel	Aucune	nullable
Adresse	id_adresse	Identifiant de la adresse	entier	PK	non nul
Adresse	adresse_numero	Numéro de l'adresse	text	Aucune	nullable
Adresse	adresse_suffixe	Suffixe de l'adresse	text	Aucune	nullable
Adresse	adresse_code_voie	Code de la voie	text	Aucune	nullable
Adresse	adresse_nom_voie	Nom de la voie	text	Aucune	nullable
Adresse	code_postal	Code postal	text	Aucune	nullable
Commune	code_commune	Code INSEE de la commune	text	FK	non nul
Commune	nom_commune	Nom de la commune	text	Aucune	non nul
Commune	ancien_code_commune	Ancien code de commune	text	Aucune	nullable
Commune	ancien_nom_commune	Ancien nom de commune	text	Aucune	nullable
Commune	code_departement	Code du département	text	FK	non nul
Parcelle	id_parcelle	Identifiant de la parcelle	text	FK	non nul
Parcelle	ancien_id_parcelle	Ancien identifiant de parcelle	text	Aucune	nullable
Parcelle	numero_disposition	Numéro du bien dans la mutation	text	Aucune	nullable
Parcelle	longitude	Longitude de la parcelle	décimal / réel	Aucune	nullable
Parcelle	latitude	Latitude de la parcelle	décimal / réel	Aucune	nullable
Local	code_type_local	Code du type de local	text	FK	non nul
Local	type_local	Libellé du type de local	text	Aucune	nullable
Culture	code_nature_culture	Code de nature de culture	text	FK	non nul
Culture	nature_culture	Libellé de nature de culture	text	Aucune	nullable
Culture_speciale	code_nature_culture_speciale	Code de nature de culture spéciale	text	FK	non nul
Culture_speciale	nature_culture_speciale	Libellé de nature de culture spéciale	text	Aucune	nullable
Lot	id_lot	Identifiant du lot	entier	FK	non nul
Lot	lot1_numero	Numéro du lot 1	text	Aucune	nullable
Lot	lot1_surface_carrez	Surface Carrez du lot 1	décimal / réel	Aucune	nullable
Lot	lot2_numero	Numéro du lot 2	text	Aucune	nullable
Lot	lot2_surface_carrez	Surface Carrez du lot 2	décimal / réel	Aucune	nullable
Lot	lot3_numero	Numéro du lot 3	text	Aucune	nullable
Lot	lot3_surface_carrez	Surface Carrez du lot 3	décimal / réel	Aucune	nullable
Lot	lot4_numero	Numéro du lot 4	text	Aucune	nullable
Lot	lot4_surface_carrez	Surface Carrez du lot 4	décimal / réel	Aucune	nullable
Lot	lot5_numero	Numéro du lot 5	text	Aucune	nullable
Lot	lot5_surface_carrez	Surface Carrez du lot 5	décimal / réel	Aucune	nullable
Lot	nombre_pieces_principales	Nombre de pièces principales	entier	Aucune	nullable
Lot	surface_reelle_bati	Surface réelle bâtie	décimal / réel	Aucune	nullable
Lot	surface_terrain	Surface du terrain	décimal / réel	Aucune	nullable
Lot	numero_volume	Numéro de volume	text	Aucune	nullable
Lot	nombre_lots	Nombre de lots	entier	Aucune	nullable

Tableau 2. Dictionnaire des variables de la base de données DVF

2.3. Analyse du fichier source

L'étude des colonnes consiste à analyser chaque champ du fichier DVF géolocalisées afin de comprendre sa signification métier, son type de données et la présence éventuelle de valeurs manquantes.

a. Etude de colonnes et types

- Colonnes relatives à la mutation
    - id_mutation (Text): identifiant textuel de la mutation foncière. Il regroupe toutes les lignes correspondant à une même mutation (vente, échange, etc.), même si plusieurs parcelles sont concernées.
    - date_mutation (Date): date au format ISO (AAAA-MM-JJ). date officielle de la signature de l'acte notarié. Elle situe l'événement dans le temps.
    - nature_mutation (Text): décrit la nature juridique de la mutation (vente, échange, adjudication, expropriation). C'est une donnée catégorielle à modalités limitées.
    - valeur_fonciere (Entier): valeur numérique de la transaction. Présence de valeurs nulles possible si la transaction ne comporte pas de prix (donation).
- Colonnes relatives à l'adresse
    - adresse_numero (Text) et adresse_suffixe (Text) : indiquent le numéro de rue et son éventuel suffixe (B, T, Q). Donnent une localisation précise de la parcelle.
    - adresse_code_voie (Text) : code FANTOIR, identifiant normalisé d'une voie au sein de la commune. Sert à relier l'adresse à une nomenclature officielle.
    - adresse_nom_voie (Text) : libellé textuel de la voie
    - code_postal (Text) : code postal normalisé sur 5 caractères. Toujours présent.
    - Colonnes relatives à la commune
    - code_commune (Text) : code INSEE de la commune (5 caractères). Toujours présent.
    - nom_commune (Text) : libellé accentué de la commune. Redondant avec code_commune, mais utile pour exploitation directe.
    - ancien_code_commune (Text) et ancien_nom_commune (Text) : colonnes de traçabilité, utiles en cas de fusion ou de changement administratif d'une commune.
- Colonnes relatives aux parcelles
    - id_parcelle (Text) : identifiant unique de la parcelle cadastrale, composé du code INSEE et des références cadastrales (14 caractères). C'est une clé importante pour relier les données au cadastre.
    - ancien_id_parcelle (Text) : référence précédente d'une parcelle après un remaniement cadastral. Permet de conserver l'historique.
    - numero_disposition (Entier): le numéro qui identifie chaque bien immobilier dans une même transaction (mutation)
    - longitude, latitude (Décimal) : coordonnées WGS-84 au centre de la parcelle. Présence de valeurs manquantes possible lorsque le géocodage n'a pas abouti (parcelle non vectorisée ou supprimée).

- Colonnes relatives aux lots
    - lot_1_numero … lot_5_numero (Entier) : numéros de lots attribués dans une copropriété. Vides si la parcelle ne relève pas d'une copropriété.
    - lot_1_surface_carrez … lot_5_surface_carrez (Décimal) : surfaces Carrez associées (décimales). Null si non concerné.
    - nombre_pieces_principales (Entier) : nombre de pièces principales sur la parcelle et le type de bâtiment. Indicateur classique du marché immobilier.
    - surface_terrain (Entier) : surface en mètres carrés (m²) de la parcelle. Null si non renseignée.
    - surface_reelle_bati (Entier) : surface construite réelle en m² sur la parcelle. Permet de différencier un appartement d'une maison, et d'évaluer sa valeur.
    - nombre_lots (Entier) : nombre total de lots concernés sur la parcelle. Peut servir de contrôle par rapport aux colonnes précédentes.
- Colonnes relatives aux locaux bâtis
    - code_type_local (Entier) : code catégoriel du type de bâtiment présent sur la parcelle (Maison, Appartement, Dépendance, etc.).
    - type_local (Text) : libellé associé au code_type_local.
- Colonnes relatives à la nature de culture
    - code_nature_culture (Text) : code décrivant la nature de la culture (terre, vigne, bois, etc.).
    - nature_culture (Text) : libellé associé.
- Colonnes relatives à la nature de culture spéciale
    - code_nature_culture_speciale (Text) : code particulier pour cultures spéciales.
    - nature_culture_speciale (Text) : libellé associé. Ces colonnes sont souvent vides si la parcelle est bâtie.

Synthèse :

- Les données sont hétérogènes : certaines colonnes ne s'appliquent qu'à certains cas (lots, culture, bâti).
- Plusieurs colonnes peuvent être nulles, nécessitant un traitement en phase ETL.
- Doublons logiques : une mutation peut concerner plusieurs parcelles/lots → redondances dans les colonnes mutation.

b. Nettoyage des données

Avant l'intégration des données dans l'architecture décisionnelle, plusieurs contrôles de qualité ont été réalisés afin d'améliorer la fiabilité des analyses futures.

- Les enregistrements dont la valeur foncière était absente ou inférieure à 1 000 € ont été supprimés afin d'écarter les transactions atypiques ou inexploitables.

- Pour les biens bâtis, les observations présentant une surface réelle bâtie inférieure à 9 m² ont également été exclues afin de respecter les critères minimaux d'habitabilité.

Ces traitements ont permis d'améliorer la qualité globale des données avant leur intégration dans le système décisionnel.

Après cette phase de nettoyage, une analyse des valeurs manquantes a été réalisée afin d'identifier les variables concernées et de comprendre l'origine de ces absences dans le jeu de données.

Valeurs manquantes

Après lecture du fichier CSV avec Python, nous pouvons identifier quelles colonnes contiennent des valeurs manquantes et comprendre pourquoi elles apparaissent.

```
import pandas as pd

# Lecture du fichier CSV
df = pd.read_csv('full.csv.gz', low_memory=False)

# Vérification des valeurs manquantes par colonne
valeurs_manquantes = df.isnull().sum()
print(valeurs_manquantes)
```

- Mutation et parcelle
    - Attributs : valeur_fonciere, numero_volume, ancien_id_parcelle, ancien_code_commune, ancien_nom_commune, longitude, latitude.
    - Raison des valeurs manquantes : certaines mutations n'ont pas de contrepartie financière (donation, succession), certaines parcelles ont été remaniées ou appartiennent à une ancienne commune. Ces informations sont donc absentes pour de nombreuses lignes.
    - Les coordonnées géographiques peuvent manquer si le géocodage n'a pas pu être effectué ou si la parcelle n'existe pas dans le cadastre vectoriel.
- Adresse
    - Attributs : adresse_numero, adresse_suffixe, adresse_nom_voie, adresse_code_voie, code_postal.
    - Raison des valeurs manquantes : pour les parcelles non bâties, les numéros et voies peuvent être absents ou non normalisés dans FANTOIR. Le code postal peut exceptionnellement manquer si l'adresse est mal renseignée.
- Local et Lot
    - Attributs : code_type_local, type_local, surface_reelle_bati, nombre_pieces_principales, lot_1_numero … lot_5_surface_carrez.
    - Raison des valeurs manquantes : si la parcelle n'est pas bâtie ou n'est pas en copropriété, ces colonnes restent vides.
- Culture
    - Attributs : code_nature_culture, code_nature_culture_speciale, nature_culture, nature_culture_speciale.
    - Raison des valeurs manquantes : les colonnes relatives à la culture ne concernent que les parcelles agricoles.

Par ailleurs, certains départements (Bas-Rhin, Haut-Rhin et Moselle) sont exclus, car ils relèvent du système du livre foncier, qui n'est pas intégré à la Base Nationale des Données Patrimoniales (BNDP). De même, Mayotte n'est pas couverte par ces données.

2.4. MCD, MLD et MPD

Nous avons construit les modèles suivants à partir du modèle de données initial de données DVF.

- **MCD (Modèle Conceptuel de Données):** Définition des entités et des relations (Figure 3). Pour la création du MCD, nous avons utilisé *Looping*, un outil qui permet de générer automatiquement le modèle conceptuel à partir des entités et associations identifiées. Cette méthode facilite la visualisation des relations entre les entités (parcelles, mutations, lots, bâtiments, cultures) et assure une cohérence dans les cardinalités et associations.

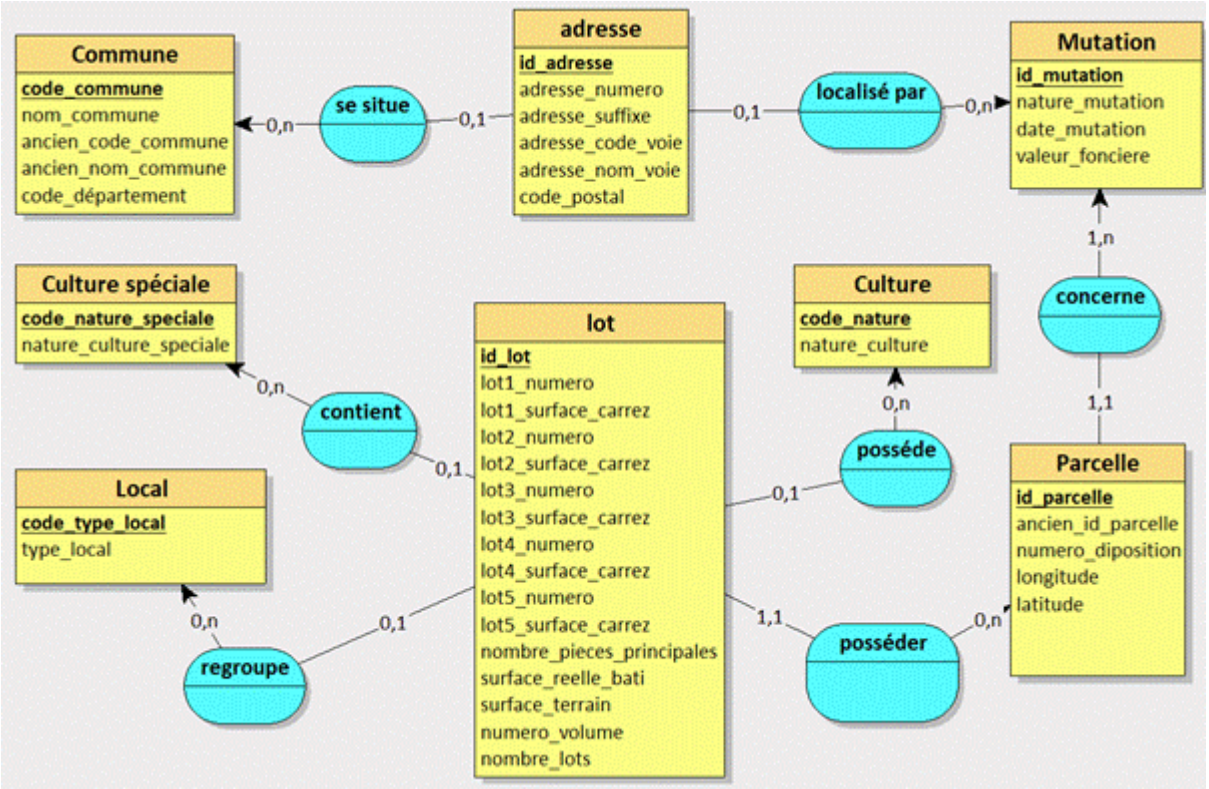


Figure 3. Modèle conceptuel des données (MCD)

- **MLD (Modèle Logique de données)**: Structuration des données sous forme relationnelle, avec identification des clés primaires(#) et étrangères (*).

Exemple:

Commune (#code_commune, nom_commune, ancien_code_commune, code_departement, ancien_nom_commune, code_departement)

Mutation (#id_mutation, nature_mutation, date_mutation, valeur_fonciere, *id_adresse)

- **MPD (Modèle Physique de Données)**: Détails sur la façon dont les données seront stockées physiquement (Figure 4). Grâce à Looping, le MCD a été converti automatiquement en MPD, ce qui permet d'obtenir directement les tables et clés primaires et étrangères nécessaires pour la base de données relationnelle.

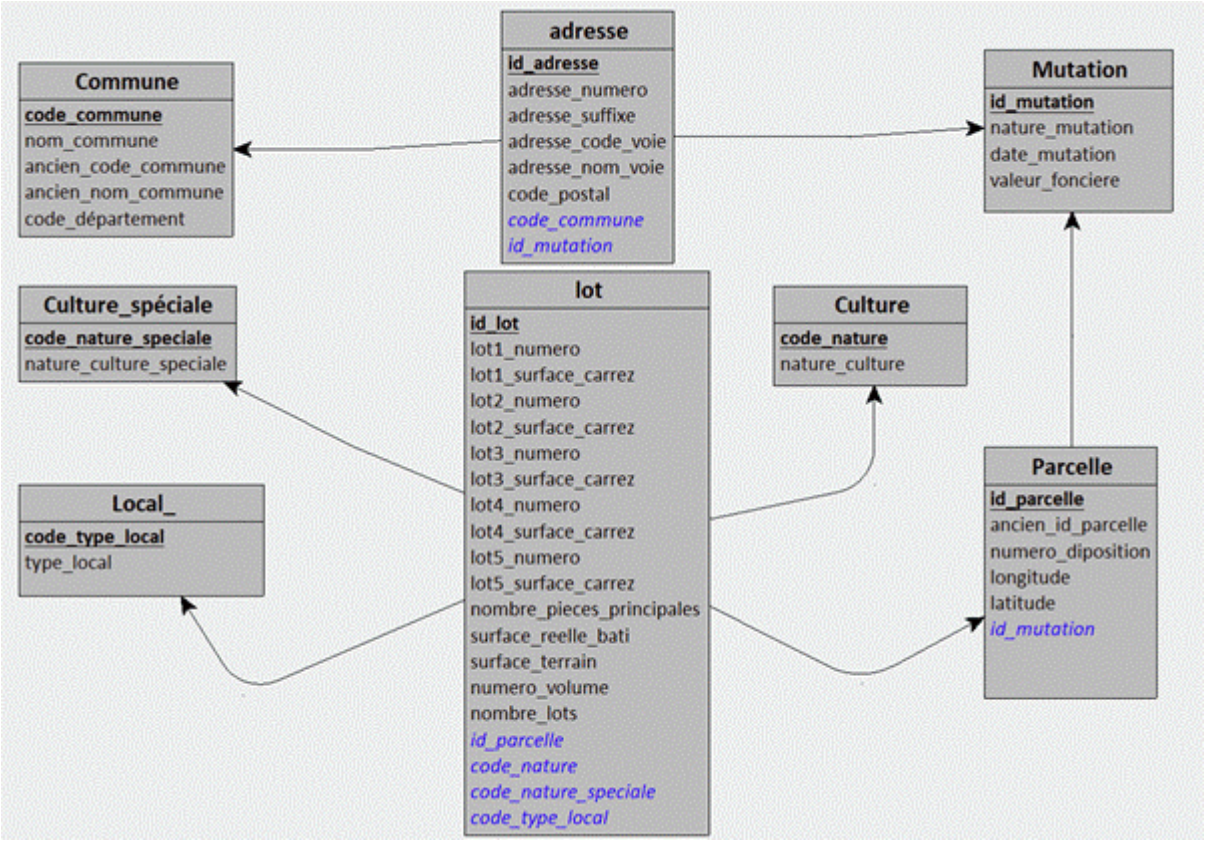


Figure 4. Modèle Physique des Données (MPD)

2.5. Scripts SQL

Grâce à Looping, un script SQL permettant de créer les tables est généré automatiquement. Nous avons ensuite repris ce script et l'avons modifié afin qu'il soit compatible avec MySQL Workbench. Le script final a été enregistré dans un fichier create_database.sql.

3. Intégration des Données

3.1. Structure DSA, ODS, DWH

A. SOURCE

La base SOURCE contient les données brutes issues des fichiers DVF fournis par la Direction Générale des Finances Publiques (DGFiP). Les données y sont intégrées sans transformation afin de conserver les informations d'origine.
Cette base regroupe les différentes tables nécessaires au projet, telles que les mutations, les parcelles, les lots, les adresses et les communes. Le modèle utilisé repose sur une approche Entité-Association permettant d'organiser les données et de définir les relations entre les différentes entités.
L'objectif de la base SOURCE est de stocker les données initiales avant leur intégration dans les bases DSA, ODS et DWH. Elle constitue ainsi le point de départ du processus d'intégration des données.

B. Data Staging Area (DSA)

Le script de création des tables pour la DSA est simple et reprend les principes utilisés pour la base SOURCE. Les contraintes appliquées sont similaires à celles de SOURCE, et le modèle utilisé est basé sur l'approche Entité-Association. Ce modèle permet d'insérer rapidement des données grâce à sa structure normalisée en évitant ainsi des duplications inutiles.

C. Operational Data Store (ODS)

Le script de création pour l'ODS suit un schéma similaire à celui du DSA, mais sans définir de contraintes entre les clés primaires et étrangères. Cela permet d'historiser les données sans contrainte. Une colonne load_date est ajoutée pour indiquer la date d'intégration des données. Le modèle utilisé reste le modèle Entité-Association, donc les données historiques sont conservées afin d'assurer la traçabilité des évolutions au fil du temps.

D. Data Warehouse (DWH)

Définition des tables

Pour construire le Data Warehouse :

- Les tables de dimension sont créées en regroupant des tables ayant un objectif similaire.
- La table de fait est identifiée comme la table contenant des indicateurs (par exemple, des mesures telles que la valeur foncière).

Modèle en étoile

Le modèle de Data Warehouse (DWH) est un modèle décisionnel, optimisé pour l'analyse et le reporting. Nous avons choisi un modèle en étoile, qui structure les données autour de tables de fait et de tables de dimensions. Ce modèle permet de :

- Minimiser les jointures complexes en simplifiant les requêtes SQL.
- Optimiser les performances, notamment pour les opérations de type SELECT.
- Centraliser les mesures importantes dans une table de faits telles que la valeur foncière.

- Permettre l’analyse selon différentes dimensions : localisation, lot, parcelle, date, type de mutation et type de bien immobilier.

Les tables de fait contiennent les données de mesure (comme les montants ou les quantités), tandis que les tables de dimensions fournissent des informations descriptives, comme des détails sur les produits, les clients ou le temps. La relation entre les deux est établie par des clés techniques (TK) dans les tables de dimension et une clé de fait dans la table centrale.

Le modèle en étoile est particulièrement efficace pour les requêtes analytiques, car il permet un accès rapide aux données sans la surcharge des jointures multiples.

Structure des dimensions
Les dimensions contiennent une clé technique (TK) qui remplace la clé primaire (NPK). Cela permet de gérer les changements sur la clé naturelle sans violer son unicité. Pour implémenter la gestion des changements (SCD Type 2), des colonnes supplémentaires ont été ajoutées :

- DateDébut : Indique le début de la validité de la ligne.
- DateFin : Initialisée à une date lointaine (09/09/2099) puis mise à jour lorsqu’une nouvelle version de l’enregistrement est créée.
- CurrentFlag : Identifie la ligne active (1 pour actif, 0 pour inactif).

Automatisation des clés techniques
Une séquence a été mise en place pour générer automatiquement les clés techniques des différentes tables du Data Warehouse. Les clés TK_mutation, TK_adresse, TK_lot, TK_parcelle ainsi que id_fait sont attribuées automatiquement lors de l’insertion de nouvelles données. Cela garantit l’unicité des enregistrements et facilite les relations entre la table de fait et les tables de dimension.

Table de gestion des dates
Une table spécifique a été créée pour gérer les dates. Elle permet d'extraire des informations telles que le trimestre, le mois, ou le jour, facilitant ainsi les analyses temporelles.

Schémas proposés
Les tables ont été regroupées pour structurer le modèle en étoile autour de ses différentes dimensions et de sa table de faits (Figure 5).

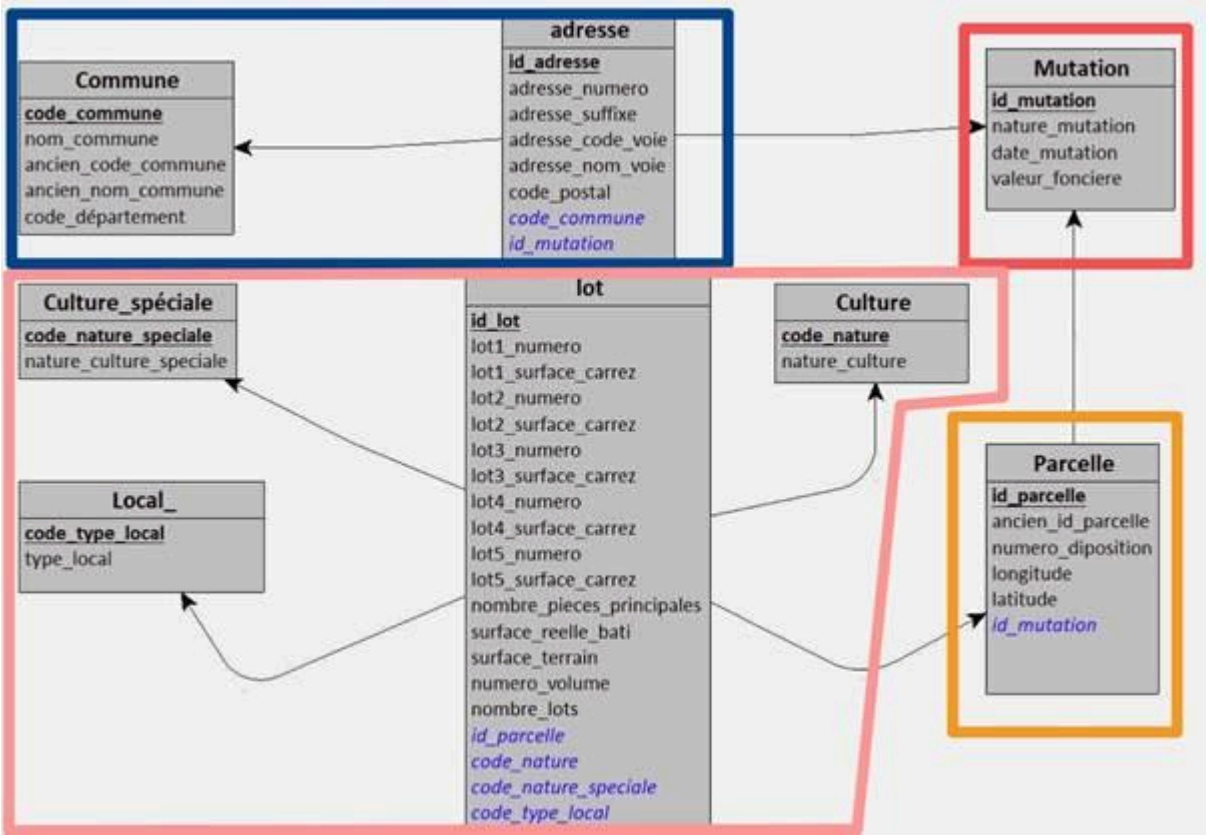


Figure 5. Regroupement des tables vers un schéma en étoile

À partir de l'analyse du modèle relationnel DVF et des besoins de reporting identifiés, nous avons conçu un modèle décisionnel en étoile permettant d'optimiser les requêtes analytiques et la visualisation des données dans Power BI.

- Table de faits :
  - Fait_DVF : contient les mesures principales relatives aux transactions immobilières, notamment la valeur foncière.
- Tables dimensionnelles :
  - Dim_Localisation : regroupe les informations géographiques (communes, départements, codes postaux) permettant d'analyser la répartition spatiale des transactions immobilières.
  - Dim_Lot : contient les caractéristiques physiques des biens (surface, nombre de pièces, type de local).
  - Dim_Date : facilite l'analyse temporelle des transactions selon l'année, le trimestre, le mois ou le jour .
  - Dim_Mutation : décrit la nature juridique des transactions immobilières.
  - Dim_Parcelle : rassemble les informations cadastrales et géographiques associées aux parcelles concernées.

La variable valeur_fonciere, initialement présente dans la table Mutation, a été déplacée vers la table de faits. Ce choix respecte les principes de la modélisation décisionnelle, car cette variable représente une mesure quantitative destinée à être agrégée et analysée selon les différentes dimensions du modèle.

Basé sur les analyses précédentes, le modèle décisionnel en étoile a été conçu à l'aide de l'application Looping (Figure 6). Cet outil permet de générer directement les scripts SQL à partir du modèle, facilitant ainsi son implémentation dans le SGBDR et garantissant la cohérence avec la base physique.

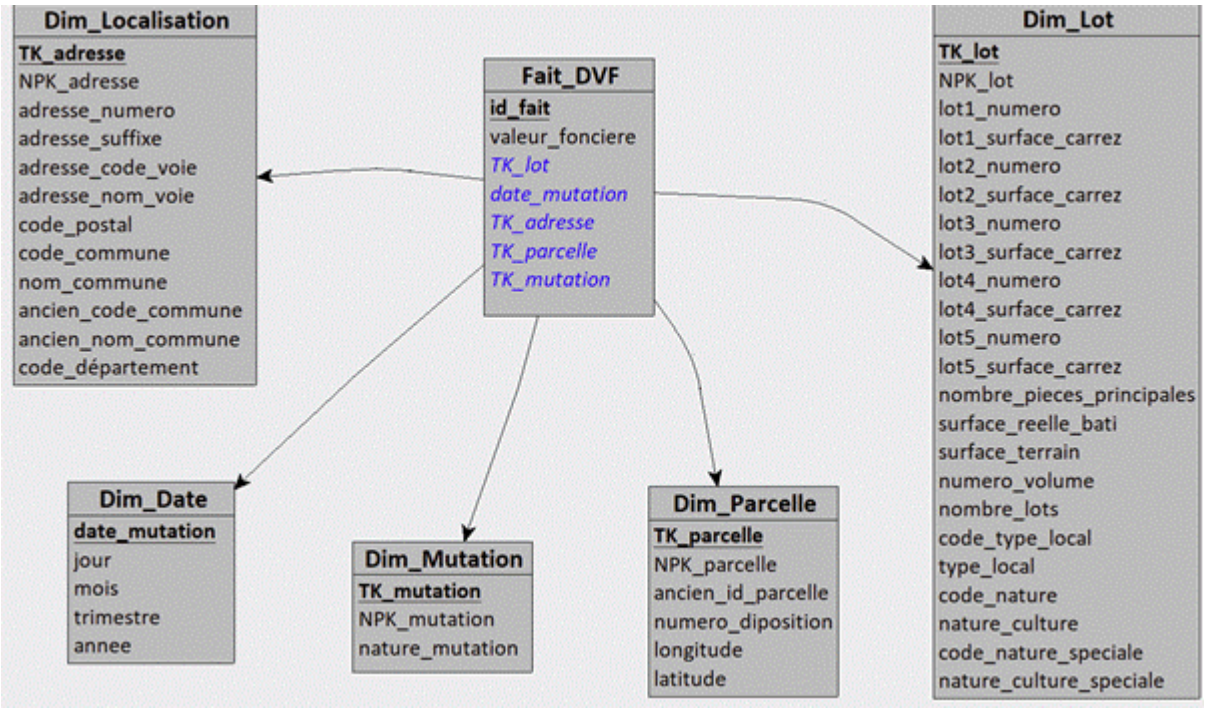


Figure 6. Schéma en étoile

3.2. Création des flux de données

Nous avons choisi Python pour notre processus ETL en raison de sa flexibilité et de sa rapidité de mise en œuvre. Cet outil nous a permis d'automatiser l'extraction, le nettoyage, la transformation et le chargement des données dans la base décisionnelle.

L'objectif principal de ce pipeline est d'automatiser les différentes étapes de traitement des données afin de préparer un Data Warehouse fiable et exploitable pour l'analyse décisionnelle et la visualisation dans Power BI.

Le pipeline permet de récupérer les données depuis les fichiers DVF.csv, de nettoyer les informations, de transformer les données selon le modèle relationnel du projet puis de charger progressivement les données dans les différentes couches du système décisionnel : SOURCE, DSA, ODS et DWH.

A. Préparation de l'environnement

Le pipeline ETL a été développé sous Jupyter Notebook, un environnement idéal pour exécuter les traitements étape par étape et contrôler les données en temps réel. Le script débute par l'installation et l'importation des bibliothèques clés :

- pandas pour lire et manipuler les fichiers CSV DVF.
- SQLAlchemy et mysql.connector permettent de connecter Python aux bases de données MySQL du projet.
- datetime pour gérer les dates de chargement et l'historisation des données.

Ensuite, la fonction **connect_engine_db(nom_db)** est créée pour automatiser la connexion aux différentes bases de données du projet (SOURCE, DSA, ODS et DWH) afin d'accéder aux données et de sécuriser les transitions entre chaque étape du pipeline ETL (Figure 7).

```
# Function : Établit une connexion à une base de données MySQL
# paramètre : nom de la base de données
# return : moteur (engine) SQLAlchemy de la base de données
def connect_engine_db(nom_db):
    try:
        # Création de l'URL de connexion pour le socket Unix
        url = f"mysql+mysqlconnector://root:NgocLam2301@/{nom_db
        # Initialisation de l'engine
        engine = create_engine(url)

        # Vérifier si la connexion fonctionne réellement,
        with engine.connect() as connection:
            connection.execute(text("SELECT 1"))

        print(f"MOTEUR PRÊT ET VÉRIFIÉ : {nom_db}")
        return engine
```

Figure 7. Code des fonctions de flux d'intégration (ex. de SOURCE vers DSA)

Plusieurs fonctions spécifiques ont ensuite été développées pour automatiser les opérations de nettoyage, de contrôle et de chargement des données. Cette approche permet de produire un code modulaire, réutilisable et conforme au principe DRY (*Don't Repeat Yourself*). Par exemple, la fonction truncate_tab() utilisée pour vider les tables avant l'intégration des données est présentée dans les Figures 8 et 9.

```
def truncate_tab(engine_db, table):
    with engine_db.connect() as conn:
        # turn off la verification la cle etranger
        conn.execute(text("SET FOREIGN_KEY_CHECKS = 0;"))
        conn.execute(text(f"TRUNCATE TABLE {table}"))
        # turn on la verification la cle etranger
        conn.execute(text("SET FOREIGN_KEY_CHECKS = 1;"))
        # commmit pour sauvegarder
        conn.commit()
```

Figure 8. Code des fonctions de flux d'intégration (ex. de SOURCE vers DSA)

```
# truncate les donnees avant inserer des donnees
truncate_tab(engine_db, tab)
```

Figure 9. Appel de la fonctions de flux d'intégration (ex. de SOURCE vers DSA)

B. Entre fichier CSV et SOURCE

Nous avons développé un script structuré en plusieurs fonctions, dont l'enchaînement logique permet d'intégrer le fichier CSV DVF dans la base SOURCE (Figure 10).

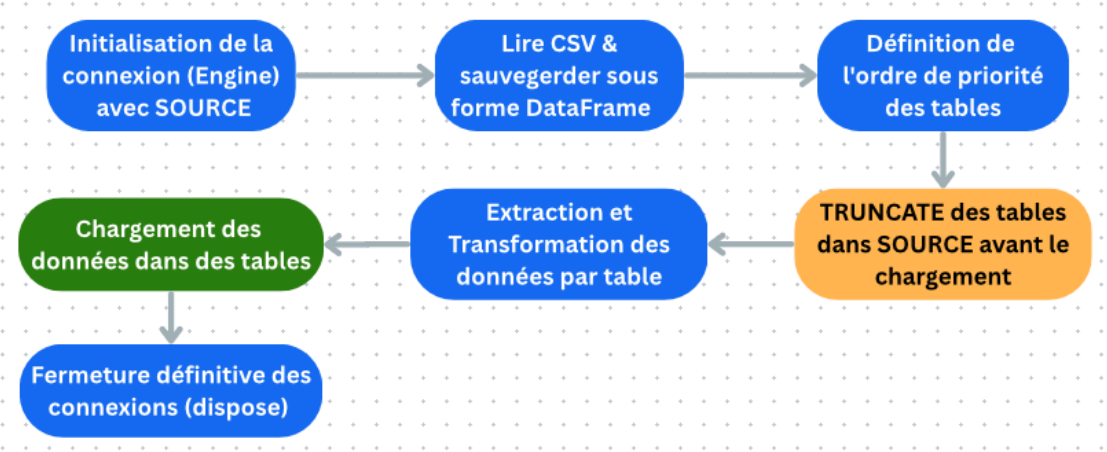


Figure 10. Diagrammes de DVF.csv vers SOURCE

Ce processus s'articule autour de plusieurs étapes pour garantir le respect des contraintes de la base de données :

- **Initialisation et connexion à la base de données SOURCE:** La fonction *connect_engine_db()* permet d'établir une connexion sécurisée avec la base de données SOURCE. Cette fonction automatise l'accès à la base et intègre un test préliminaire afin de vérifier que le serveur est prêt à recevoir les données.
- **Lecture et découpage du fichier CSV**: La fonction *csv_df()* permet de charger le fichier CSV brut sous forme d'un DataFrame. À cette étape, les données sont lues et certaines valeurs manquantes ou incohérentes, notamment concernant la valeur foncière, sont traitées. La fonction *sql_df_db_source()* permet ensuite de répartir les données du DataFrame dans les différentes tables de la base SOURCE en fonction des colonnes correspondant à chaque entité.
- **Gestion des contraintes d'intégrité (Clés étrangères)**: La base SOURCE doit respecter un ordre strict de chargement afin d'éviter toute violation des contraintes de clés étrangères. La fonction *tab_priority()* génère une liste de priorité permettant d'alimenter les tables parentes (Commune, Mutation) avant les tables enfants (Parcelle, Lot, Adresse).
- **Respect de la Clé Primaire**: Le fichier CSV contient de nombreuses répétitions. Une même mutation immobilière peut apparaître sur plusieurs lignes lorsqu'elle concerne plusieurs parcelles ou plusieurs lots. Afin d'éviter les violations de clés primaires, la fonction *sql_df_db_source()* applique un dédoublonnement sur les identifiants métiers tels que id_mutation ou id_parcelle avant l'importation.
- **Truncate des tables**: La fonction truncate_tab() vide les tables cibles avant chaque nouveau chargement afin d'éviter l'accumulation de doublons lors des exécutions successives du pipeline.
- **Importation et vérification finale**: La fonction *sql_df_db_source()* insère les données dans les tables correspondantes de la base SOURCE à l'aide de la méthode *to_sql()*. Enfin, la fonction *verifier_db_tab()* permet de contrôler le nombre de lignes insérées ainsi que la cohérence globale du chargement.

- **Fermeture des connexions:** Afin d'éviter que les opérations réalisées n'affectent les données ou les ressources des bases après le chargement, les connexions avec les bases SOURCE sont fermées à l'aide de la méthode dispose().

C. Entre SOURCE et DSA

Après l'intégration des données dans la base SOURCE, celles-ci sont transférées vers la couche DSA. Ce processus est décrit dans la Figure 11.

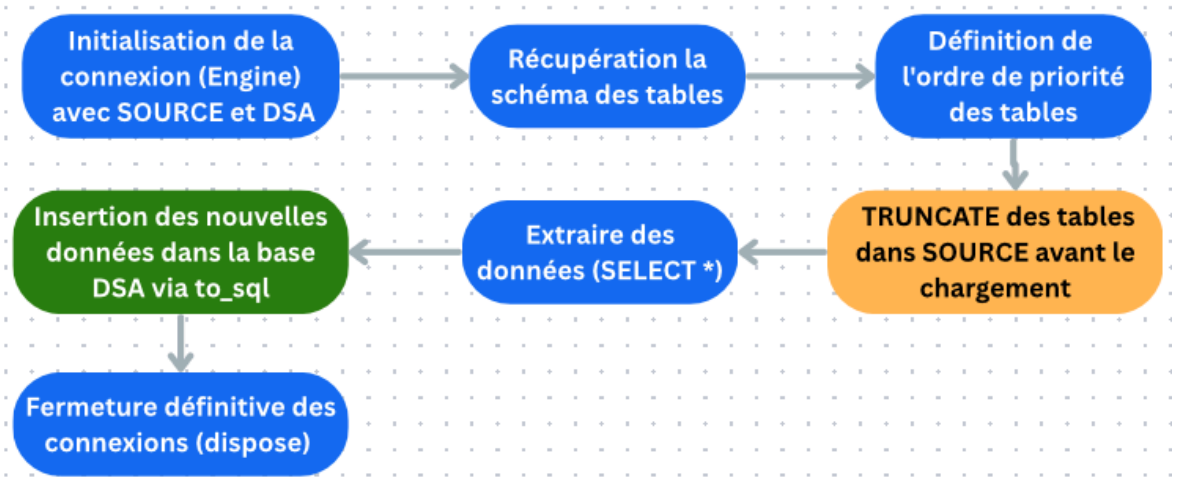


Figure 11. Diagrammes de SOURCE vers DSA

Ce processus s'articule autour de plusieurs étapes pour garantir le respect des contraintes de la base de données :

- **Initialisation et connexion à la base de données SOURCE et DSA:** La fonction connect_engine_db() permet d'établir les connexions avec les bases de données SOURCE et DSA. La base SOURCE contient les données d'origine et la base DSA sert de zone intermédiaire pour le chargement des données
- **Gestion des contraintes d'intégrité (Clés étrangères)**: La base DSA doit respecter un ordre strict de chargement afin d'éviter toute violation des contraintes de clés étrangères. La fonction *tab_priority()* génère une liste de priorité permettant d'alimenter les tables parentes (Commune, Mutation) avant les tables enfants (Parcelle, Lot, Adresse).
- **Truncate des tables**: Avant chaque nouveau chargement, la fonction truncate_tab() vide les tables cibles de la base DSA. Cette étape permet de garantir que la DSA contient uniquement les nouvelles données issues de la base SOURCE.
- **Export des données:** Pour chaque table de la base SOURCE, les données sont extraites à l'aide d'une requête SQL "*SELECT * FROM table*". Les résultats obtenus sont stockés sous forme de DataFrame pour faciliter leur insertion dans la base DSA.
- **Importation et vérification finale**: Les DataFrames précédemment créés sont insérés dans les tables correspondantes de la base DSA à l'aide de la méthode to_sql(). Enfin, la fonction verifier_db_tab() permet de contrôler le nombre de lignes insérées ainsi que la cohérence globale du chargement.

- **Fermeture des connexions:** Afin d'éviter que les opérations réalisées n'affectent les données ou les ressources des bases après le chargement, les connexions avec les bases SOURCE et DSA sont fermées à l'aide de la méthode dispose().

D. Entre DSA et ODS

Lors du transfert des données entre la DSA et l'ODS, il n'est pas nécessaire de respecter un ordre strict lié aux contraintes d'intégrité, car les tables de l'ODS ne contiennent pas de contraintes de clés primaires ni de clés étrangères.

L'objectif principal de cette étape est de conserver un historique des données. Ainsi, les données ne sont pas simplement remplacées, mais enregistrées afin de suivre leurs évolutions au fil du temps. La Figure 12 illustre ce processus.

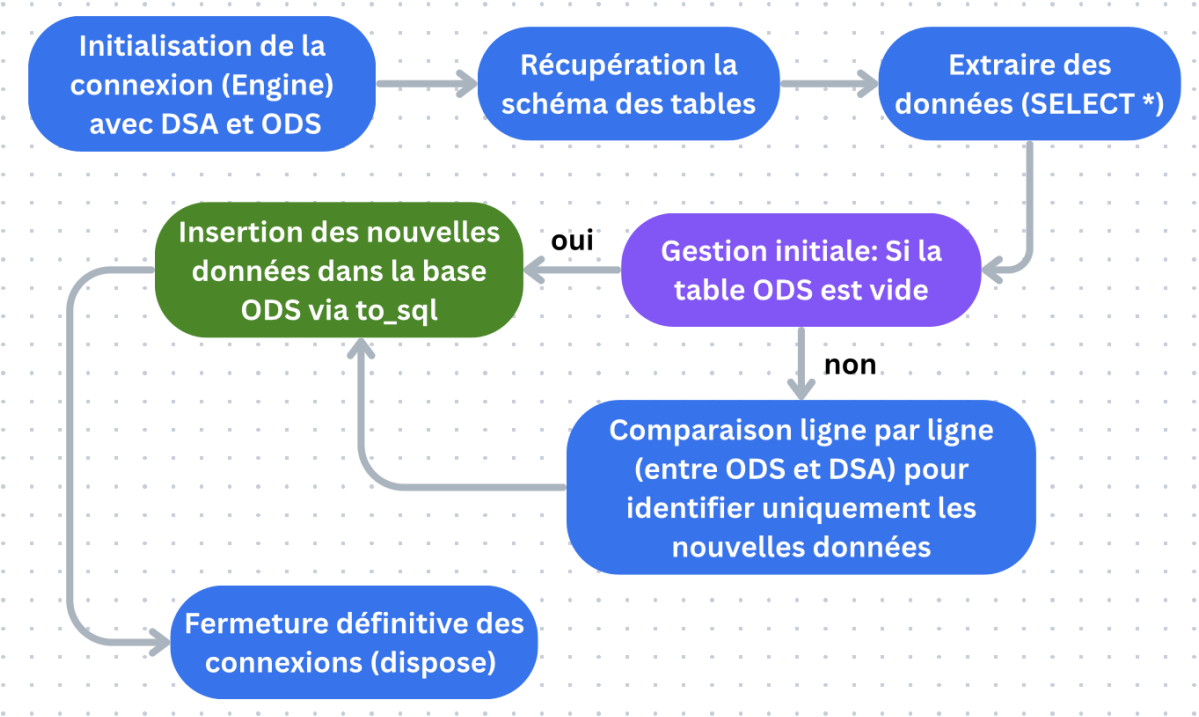


Figure 12. Diagrammes de DSA vers ODS

Ce processus s'articule autour de plusieurs étapes pour garantir le respect des contraintes de la base de données :

- **Initialisation et connexion à la base de données DSA et ODS:** La fonction connect_engine_db() permet d'établir les connexions avec les bases de données DSA et ODS. La base DSA contient les données d'origine et la base ODS sert de zone intermédiaire pour le chargement des données
- **Export des données:** Pour chaque table de la base DSA, les données sont extraites à l'aide d'une requête SQL "*SELECT * FROM table*". Les résultats obtenus sont stockés sous forme de DataFrame_1 pour faciliter leur insertion dans la base ODS.

- **Lecture des données existantes dans l'ODS:** Le programme lit d'abord la table correspondante dans la base ODS. Si cette table est vide, les données extraites depuis la DSA sont directement insérées dans l'ODS avec to_sql(). Dans le cas contraire, les données déjà présentes dans l'ODS sont stockées dans un DataFrame_2 afin d'être utilisées pour la comparaison. Lorsqu'un même identifiant possède plusieurs lignes dans l'ODS, seule la ligne ayant la *load_date* la plus récente est conservée, car elle représente la dernière version connue de la donnée.
- **Comparaison et sélection des données à historiser:** Les données de la DSA sont ensuite comparées avec celles déjà présentes dans l'ODS. Cette comparaison permet d'identifier les nouvelles données, c'est-à-dire les identifiants qui n'existent pas encore dans l'ODS, ainsi que les données dont les valeurs ont changé. Ces lignes sont enregistrées dans un DataFrame_3, car elles correspondent à de nouvelles informations ou à des évolutions à conserver dans l'historique.
- **Importation et vérification finale**: Les DataFrames précédemment sont insérés dans les tables correspondantes de la base ODS à l'aide de la méthode *to_sql()*. Enfin, la fonction *verifier_db_tab()* permet de contrôler le nombre de lignes insérées ainsi que la cohérence globale du chargement.
- **Fermeture des connexions:** Afin d'éviter que les opérations réalisées n'affectent les données ou les ressources des bases après le chargement, les connexions avec les bases DSA et ODS sont fermées à l'aide de la méthode *dispose()*.

E. Entre ODS et DWH

La dernière étape du pipeline consiste à alimenter le Data Warehouse à partir des données historisées présentes dans l'ODS (Figure 13). Cette phase a pour objectif de transformer les données afin de respecter le modèle décisionnel en étoile défini précédemment.

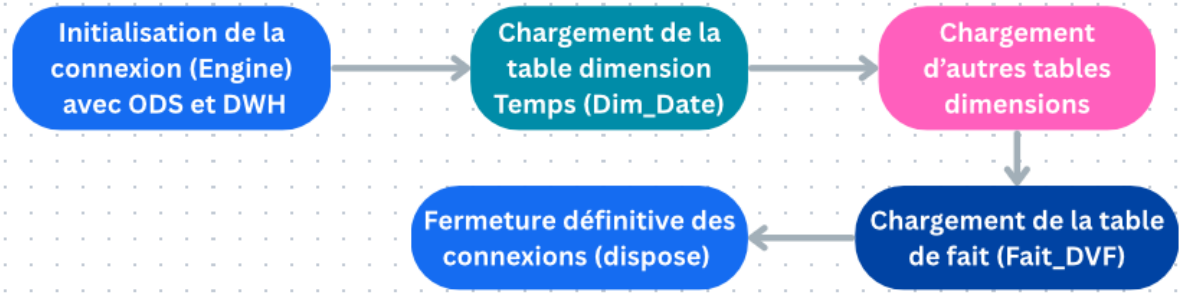


Figure 13. Diagrammes de ODS vers DWH

Ce processus s'articule autour de plusieurs étapes pour garantir le respect des contraintes de la base de données :

- **Initialisation et connexion à la base de données ODS et DWH:** La fonction connect_engine_db() permet d'établir les connexions avec les bases de données DSA et ODS. La base DSA contient les données d'origine et la base ODS sert de zone intermédiaire pour le chargement des données

- **Fermeture des connexions:** Enfin, la fonction verifier_db_tab() permet de contrôler le nombre de lignes insérées ainsi que la cohérence globale du chargement. Afin d'éviter que les opérations réalisées n'affectent les données ou les ressources des bases après le chargement, les connexions avec les bases ODS et DWH sont fermées à l'aide de la méthode dispose().

Le chargement débute par l'alimentation de la dimension temporelle Dim_Date (Figure 14). Une plage de dates est générée automatiquement afin de permettre les analyses temporelles selon l'année, le trimestre, le mois ou le jour.

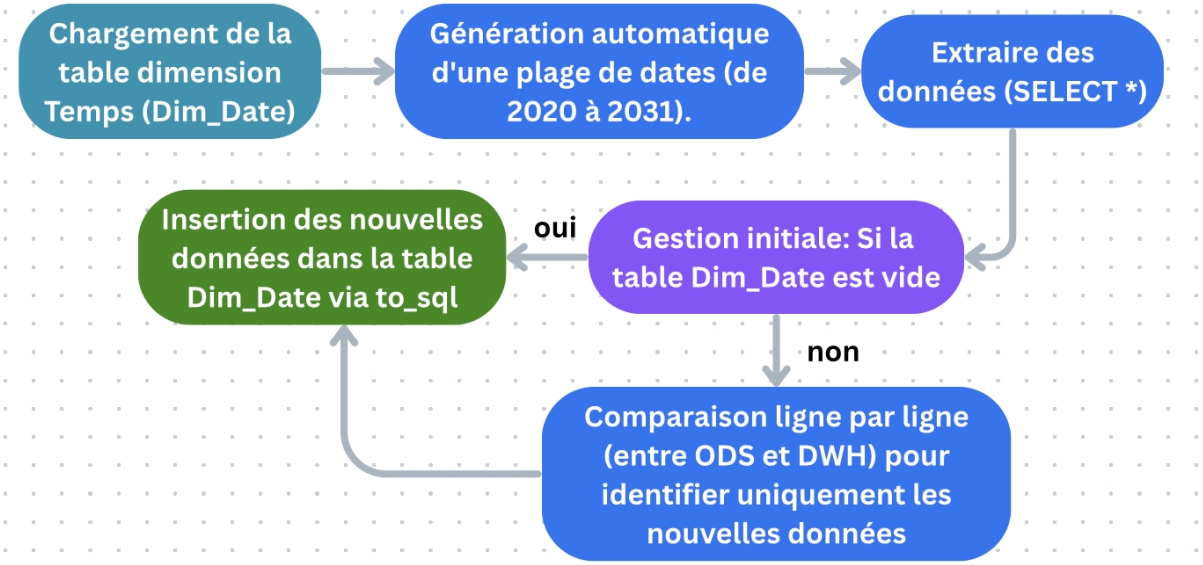


Figure 14. Diagrammes de la table dimension (Dim_Date)

Ce processus s'articule autour de plusieurs étapes pour garantir le respect des contraintes de la base de données :

- **Génération automatique des dates:** Le programme génère automatiquement les dates de l'année 2020 jusqu'à l'année 2031. Ces données sont ensuite transformées sous forme de DataFrame_1.
- **Lecture des données existantes dans la table Dim_Date**: Cette table est lue à l'aide de la requête "*SELECT * FROM Dim_Date*". Les résultats obtenus sont stockés dans un DataFrame_2 de so sanh voi dataframe Dim_Date.Les résultats obtenus sont stockés sous forme de DataFrame_2 pour faciliter leur insertion dans la table Dim_Date comparer avec le DataFrame_1 générées précédemment.
- **Comparaison des données**: Si le DataFrame_2 est vide, le DataFrame_1 est directement inséré dans la table à l'aide de la méthode to_sql(). Dans le cas contraire, le programme compare les données du DataFrame_1 avec celles du DataFrame_2. Les dates présentes dans le DataFrame_1, mais absentes du DataFrame_2, sont sélectionnées et stockées dans un DataFrame_3.
- **Importation finale**: Les données contenues dans le DataFrame_3 sont ensuite insérées dans la table Dim_Date à l'aide de la méthode to_sql().

Les autres dimensions du Data Warehouse sont ensuite alimentées (Figure 15). Lorsqu'une modification est détectée, le mécanisme de Slowly Changing Dimension de type 2 (SCD Type 2) est appliqué.

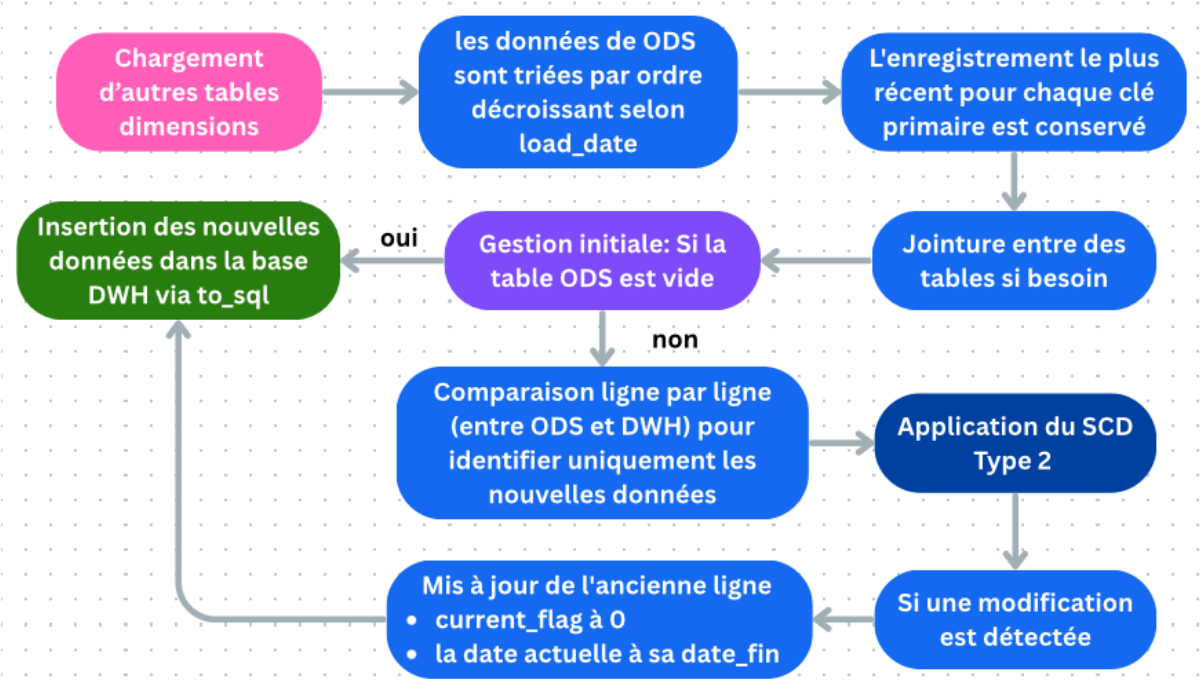


Figure 15. Diagrammes d'autres tables dimensions

Ce processus s'articule autour de plusieurs étapes pour garantir le respect des contraintes de la base de données :

- **Lecture des données ODS:** Le programme lit d'abord les tables de l'ODS nécessaires à la construction de la dimension concernée. Les données sont filtrées pour conserver une seule ligne par identifiant. Si un même identifiant possède plusieurs lignes, seule la ligne ayant la *load_date* la plus récente est gardée. Si nécessaire, plusieurs tables ODS sont jointes entre elles. Par exemple, la dimension *Dim_Localisation* est construite à partir des tables *ODS_Adresse* et *ODS_Commune*. Le résultat est stocké dans un DataFrame_1
- **Lecture de la dimension DWH**: Le programme lit ensuite la table de dimension correspondante dans le DWH. Les données existantes sont stockées dans un DataFrame_2. Si cette table est vide, le DataFrame_1 est directement inséré dans la table du DWH avec la méthode to_sql().
- **Comparaison des données:** Si la table de dimension contient déjà des données, le programme compare le DataFrame_1 avec le DataFrame_2. Les lignes présentes dans le DataFrame_1, mais absentes du DataFrame_2, sont stockées dans un DataFrame_3. Les lignes déjà existantes mais concernées par une modification sont stockées dans un DataFrame_4.
- **Mise à jour de l'historique:** Les lignes du DataFrame_2 correspondant au DataFrame_4 sont mises à jour dans le DWH. Leur champ current_flag passe à "0" et leur champ date_fin prend la date du jour, afin d'indiquer qu'elles ne sont plus les versions actuelles.

- **Insertion des nouvelles versions**: Enfin, les données du DataFrame_3 sont insérées dans la table de dimension correspondante avec la méthode to_sql(). Les colonnes *current_flag*, *date_debut* et *date_fin* ayant déjà des valeurs par défaut, elles ne nécessitent pas de modification manuelle.

Enfin, la table de faits Fait_DVF est alimentée à partir des données présentes dans l'ODS et des clés techniques récupérées dans les différentes dimensions (Figure 16.1). Les jointures nécessaires sont réalisées afin d'obtenir les clés substituts (TK) associées aux dimensions. Les nouvelles lignes sont ensuite insérées dans la table de faits tout en garantissant la cohérence avec le modèle en étoile.

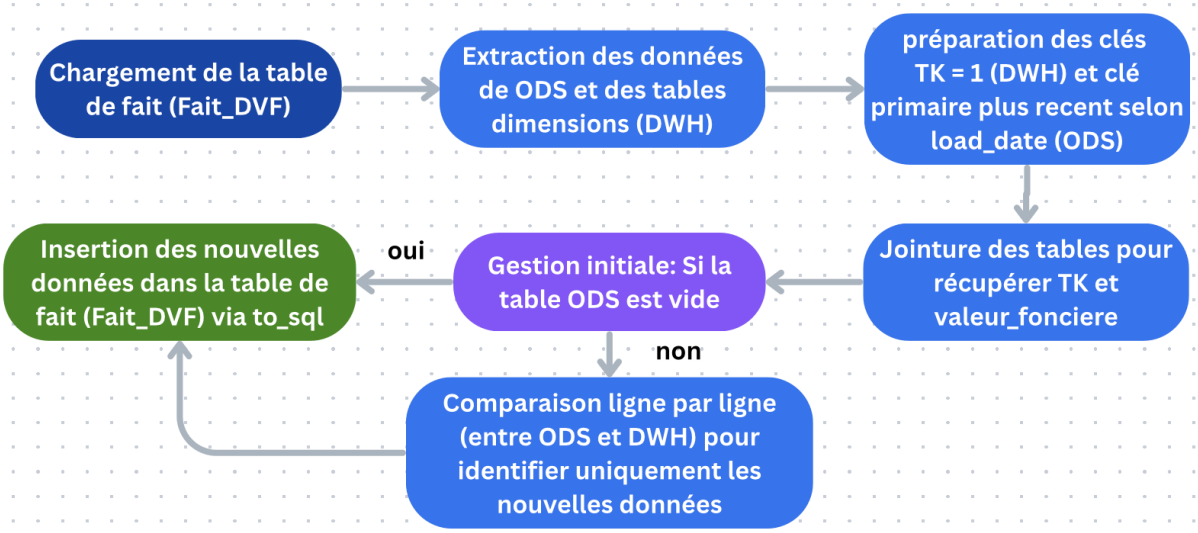


Figure 16.1. Diagrammes de la table de fait (Fait_DVF)

Ce processus s'articule autour de plusieurs étapes pour garantir le respect des contraintes de la base de données :

- **Lecture des données:** Le programme lit d'abord les données nécessaires dans les tables correspondantes de l'ODS ainsi que dans les tables de dimensions du DWH, comme illustré dans la Figure 16.1. Pour les tables de dimensions, seules les lignes correspondant à la version actuelle sont prises en compte, sauf pour la table Dim_Date.
- **Jointure des tables**: Les différentes tables sont ensuite jointes afin de récupérer les clés techniques des dimensions, ainsi que la mesure valeur_fonciere provenant de l'ODS. Le résultat obtenu est stocké dans un DataFrame_1.
- **Lecture de la table de faits**: La table Fait_DVF existante dans le DWH est ensuite lue et stockée dans un DataFrame_2. Si ce DataFrame est vide, cela signifie que la table de faits ne contient encore aucune donnée. Dans ce cas, le DataFrame_1 est directement inséré dans Fait_DVF à l'aide de la méthode to_sql().
- **Contrôle des doublons:** Si le DataFrame_2 contient déjà des données, le programme compare le DataFrame_1 avec le DataFrame_2. Les lignes présentes dans le DataFrame_1, mais absentes du DataFrame_2, sont sélectionnées et stockées dans un DataFrame_3.

- **Insertion finale:** Enfin, le DataFrame_3 est inséré dans la table Fait_DVF du DWH avec la méthode to_sql().



Figure 16.1. Schéma des jointures

F. Fonction principale du pipeline ETL

Enfin, une fonction principale (Figure 17) a été développée afin d'exécuter automatiquement l'ensemble des flux du pipeline ETL en une seule opération. Cette fonction permet d'enchaîner successivement les étapes d'intégration entre les différentes couches (SOURCE, DSA, ODS et DWH) sans intervention manuelle.

```python
def flux_total_sae():
    # flux source CSV vers Source database
    flux_csv_source()
    # flux Source database vers DSA
    flux_source_dsa()
    # flux DSA vers ODS
    flux_dsa_ods()
    # flux ODS vers DWH
    flux_ods_dwh()
```

Figure 17. Structure de la fonction principale (main) pour l'orchestration du pipeline

3.3. Validation et Optimisation

Après chaque étape d'intégration, plusieurs contrôles ont été réalisés afin de garantir la qualité, la cohérence et la fiabilité des données transférées entre les différentes couches du système décisionnel.

- **Contrôle du nombre de lignes :** Des vérifications ont été effectuées à l'aide de requêtes SQL telles que COUNT(*) afin de comparer le nombre d'enregistrements présents dans les différentes bases. Par exemple, la table *Adresse* contient 44 256 lignes dans la base SOURCE ainsi que dans la base DSA (Figures 18 et 19), ce qui confirme la bonne exécution du transfert des données.
- **Validation des intégrations répétées :** Plusieurs exécutions du pipeline ont été réalisées à différents moments afin de vérifier que les résultats obtenus restent cohérents et identiques d'une exécution à l'autre. Ces tests permettent de confirmer la stabilité du processus ETL.

```
mysql> select count(*) from db_source.Adresse;
+----------+
| count(*) |
+----------+
|    44256 |
+----------+
1 row in set (0.01 sec)
```

Figure 18. Nombre de lignes de la table Adresse dans SOURCE

```
mysql> select count(*) from db_dsa.DSA_Adresse;
+----------+
| count(*) |
+----------+
|    44256 |
+----------+
1 row in set (0.05 sec)
```

Figure 19. Nombre de lignes de la table DSA_Adresse dans DSA

- **Contrôle de l'intégrité des données :** Des vérifications ont été effectuées afin de s'assurer du respect des clés primaires et des clés étrangères ainsi que de l'absence de doublons dans les tables chargées.
- **Tests sur deux machines distinctes :** Les intégrations ont été réalisées sur deux environnements différents afin de vérifier la reproductibilité des traitements et la cohérence des résultats obtenus.

L'ensemble de ces contrôles a permis de valider le bon fonctionnement du pipeline ETL et de garantir la fiabilité des données utilisées pour l'alimentation du Data Warehouse et des tableaux de bord Power BI.

4. Construction du Tableau de Bord Power BI

4.1. Définition des indicateurs de pilotage

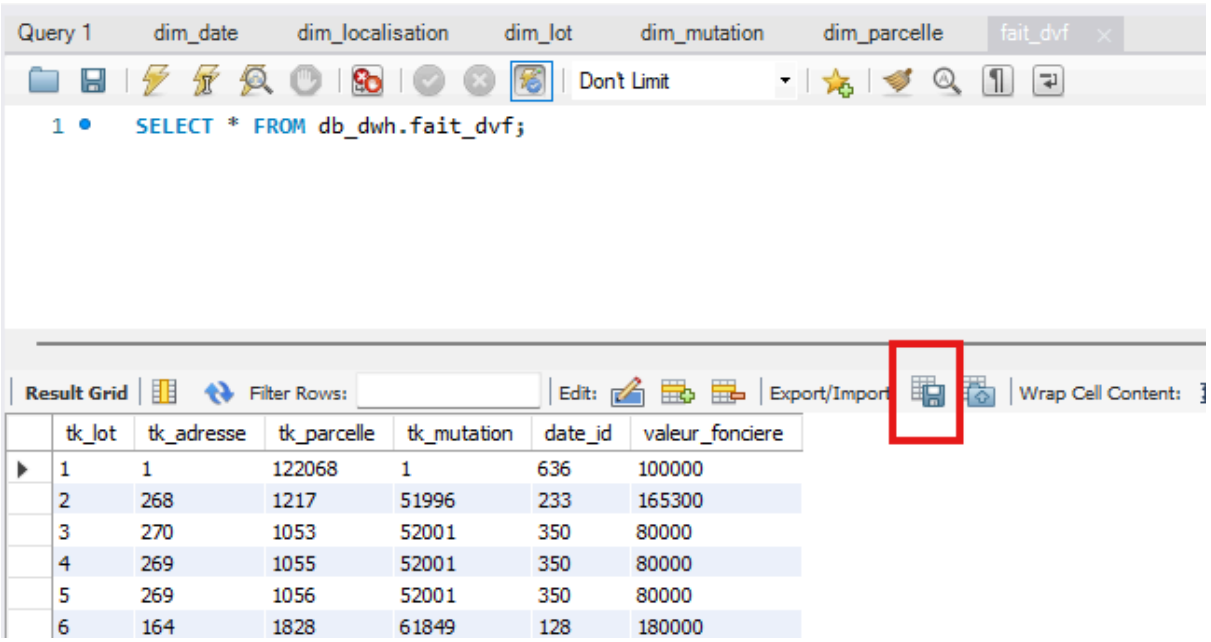
Avant de concevoir les tableaux de bord Power BI, une réflexion a été menée afin d'identifier les principaux indicateurs nécessaires à l'analyse du marché immobilier. L'objectif était de fournir aux utilisateurs une vision synthétique de l'activité à travers plusieurs indicateurs clés de performance (KPI).

Les principaux indicateurs retenus sont le nombre total de mutations, la valeur foncière totale, la valeur foncière moyenne, la valeur foncière médiane, la surface moyenne bâtie, le nombre de communes, le nombre de départements ainsi que le prix moyen au m².

Ces indicateurs permettent d'analyser l'évolution du marché immobilier, d'identifier les zones les plus dynamiques et de comparer les caractéristiques des biens selon différents critères géographiques et temporels.

4.2. Extraction des données du Data Warehouse

Une fois le processus d'intégration terminé et les données consolidées dans le Data Warehouse (DWH), nous avons procédé à leur exploitation afin de construire des tableaux de bord décisionnels sous Power BI.
Dans un premier temps, les données contenues dans les différentes tables du DWH ont été vérifiées directement dans MySQL Workbench à l'aide de requêtes SQL de consultation. Cette étape a permis de contrôler la qualité, la cohérence et l'exhaustivité des données avant leur exportation vers Power BI.
Les données ont ensuite été exportées au format CSV à l'aide de la fonctionnalité *Export Recordset* de MySQL Workbench. Ce format a été retenu pour sa compatibilité avec Power BI et sa capacité à conserver l'intégrité des données issues du Data Warehouse.
Les tables exportées sont les suivantes : Dim_Date, Dim_Mutation, Dim_Lot, Dim_Localisation, Dim_Parcelle et Fait_DVF**.**
Certaines tables, notamment Dim_Lot**,** Dim_Mutation et Fait_DVF, présentent un volume important de données. Cette volumétrie élevée peut entraîner des temps d'exportation plus longs et une consommation accrue des ressources système. Malgré cette contrainte, cette méthode garantit un transfert fiable et complet des données vers l'outil de visualisation.
La Figure 20 illustre la consultation des données de la table Fait_DVF dans MySQL Workbench ainsi que leur exportation au format CSV.



tk_lot	tk_adresse	tk_parcelle	tk_mutation	date_id	valeur_fonciere
1	1	122068	1	636	100000
2	268	1217	51996	233	165300
3	270	1053	52001	350	80000
4	269	1055	52001	350	80000
5	269	1056	52001	350	80000
6	164	1828	61849	128	180000

Figure 20. Consultation de la table Fait_DVF et exportation des données depuis MySQL Workbench

Une fois les fichiers CSV générés, ceux-ci ont été importés dans Power BI Desktop via Power Query. Cette étape a permis de préparer les données pour les phases de modélisation et de visualisation. Les opérations réalisées comprenaient notamment la vérification des types de données, le renommage de certaines colonnes, la suppression des colonnes inutiles et le contrôle de la cohérence des clés utilisées pour les relations entre les tables.

Après cette phase de préparation, les tables ont été chargées dans Power BI afin de reconstruire le schéma en étoile défini lors de la conception du Data Warehouse. La table de faits Fait_DVF a été reliée aux différentes tables dimensionnelles (Dim_Date, Dim_Mutation, Dim_Lot, Dim_Localisation et Dim_Parcelle) à l'aide de leurs clés techniques respectives. Cette modélisation facilite les analyses multidimensionnelles et améliore les performances des calculs DAX utilisés dans les tableaux de bord.

4.3. Modélisation des données dans Power BI

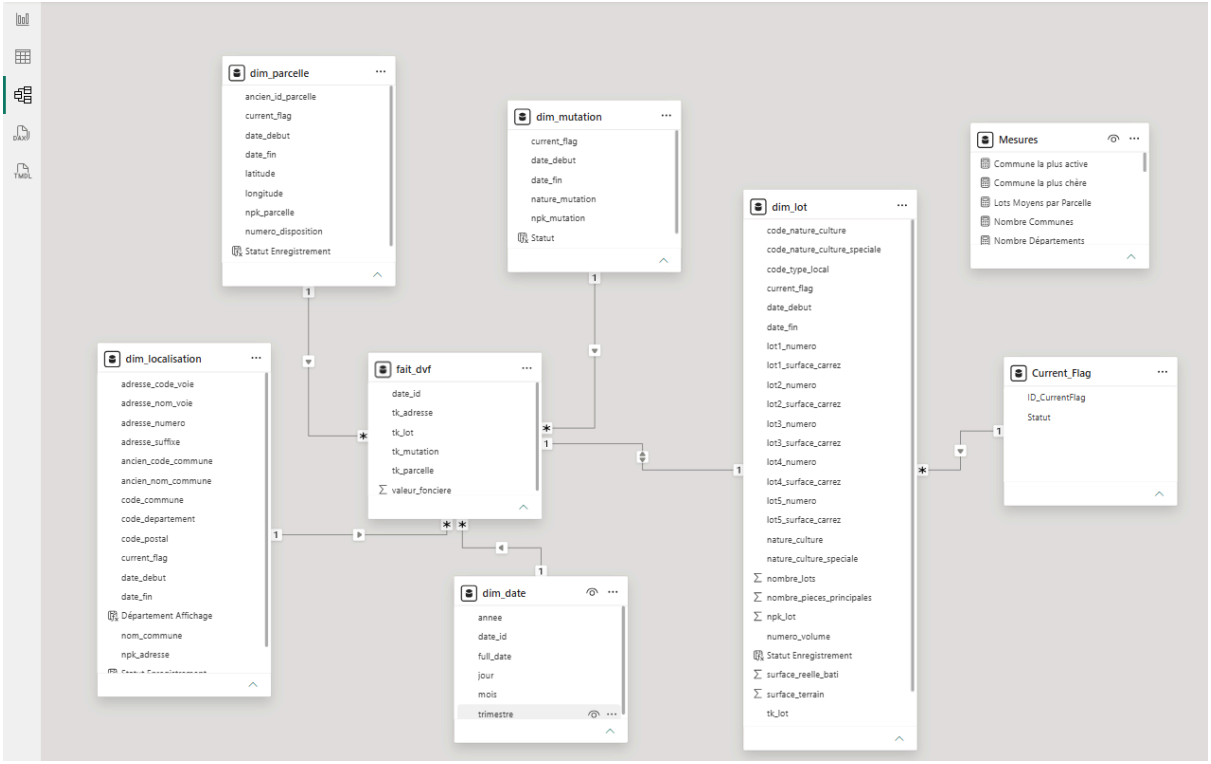


Figure 21. Modèle de données Power BI basé sur le schéma en étoile du Data Warehouse

Le modèle de données est structuré autour d'une table de faits (Fait_DVF) et de cinq tables dimensionnelles (Dim_Date, Dim_Localisation, Dim_Lot, Dim_Mutation et Dim_Parcelle). Les relations de type un-à-plusieurs (1) entre la table de faits et les dimensions permettent de réaliser des analyses multidimensionnelles performantes tout en optimisant l'exécution des calculs DAX utilisés dans les tableaux de bord.

Afin d'améliorer la lisibilité et l'interprétation des données, une table supplémentaire nommée Current_Flag a été créée. Cette table permet d'identifier rapidement le statut des enregistrements : une valeur de 1 correspond à une donnée actuellement valide, tandis qu'une valeur de 0 correspond à une donnée historique conservée dans le cadre du suivi des évolutions du Data Warehouse. Cette approche simplifie la création des filtres dans Power BI et facilite la compréhension des analyses par les utilisateurs finaux.

Par ailleurs, une table dédiée aux mesures DAX a été ajoutée au modèle. Cette table ne contient aucune donnée opérationnelle mais regroupe l'ensemble des indicateurs calculés utilisés dans les tableaux de bord. Cette organisation permet de centraliser les mesures, d'améliorer la lisibilité du modèle de données et de faciliter la maintenance ainsi que les évolutions futures du projet Power BI.

Après la modélisation du schéma de données, plusieurs mesures DAX ont été développées afin de calculer les indicateurs clés nécessaires à l'analyse du marché immobilier.

4.4. Création des mesures DAX

Afin d'enrichir l'analyse des données DVF, plusieurs mesures DAX ont été développées dans Power BI. Ces mesures permettent de calculer automatiquement les principaux indicateurs utilisés dans les différentes pages du tableau de bord.

Pour faciliter l'organisation du modèle, l'ensemble des mesures a été regroupé dans une table dédiée nommée **Mesures**. Cette approche améliore la lisibilité du modèle Power BI et simplifie la maintenance des indicateurs.

Parmi les principales mesures créées figurent la valeur foncière totale, le nombre de mutations, le prix moyen par mutation, la surface moyenne bâtie ainsi que plusieurs indicateurs relatifs aux parcelles, aux lots, aux communes et aux prix immobiliers. Ces mesures permettent d'analyser le marché immobilier selon différents axes géographiques et temporels.

La Figure 22 présente la table regroupant l'ensemble des mesures DAX utilisées dans le tableau de bord.

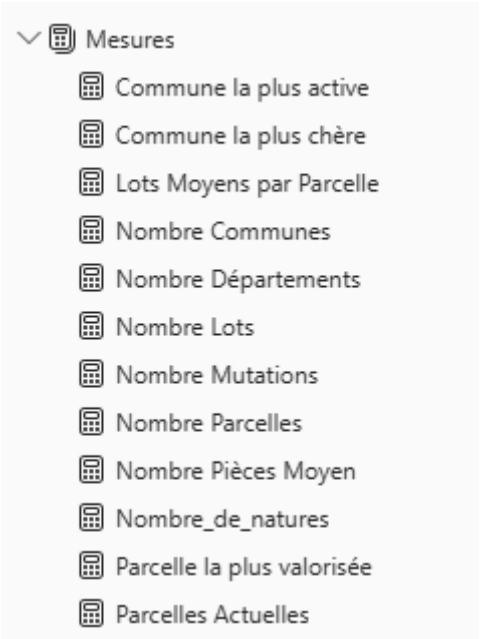


Figure 22. Table des mesures DAX utilisées dans Power BI

Au total, 24 mesures DAX ont été développées pour répondre aux différents besoins d'analyse du projet DVF. Ces mesures constituent la base des indicateurs utilisés dans l'ensemble des tableaux de bord et permettent d'automatiser les calculs décisionnels réalisés dans Power BI.

4.5. Présentation des tableaux de bord

A. Page Synthèse

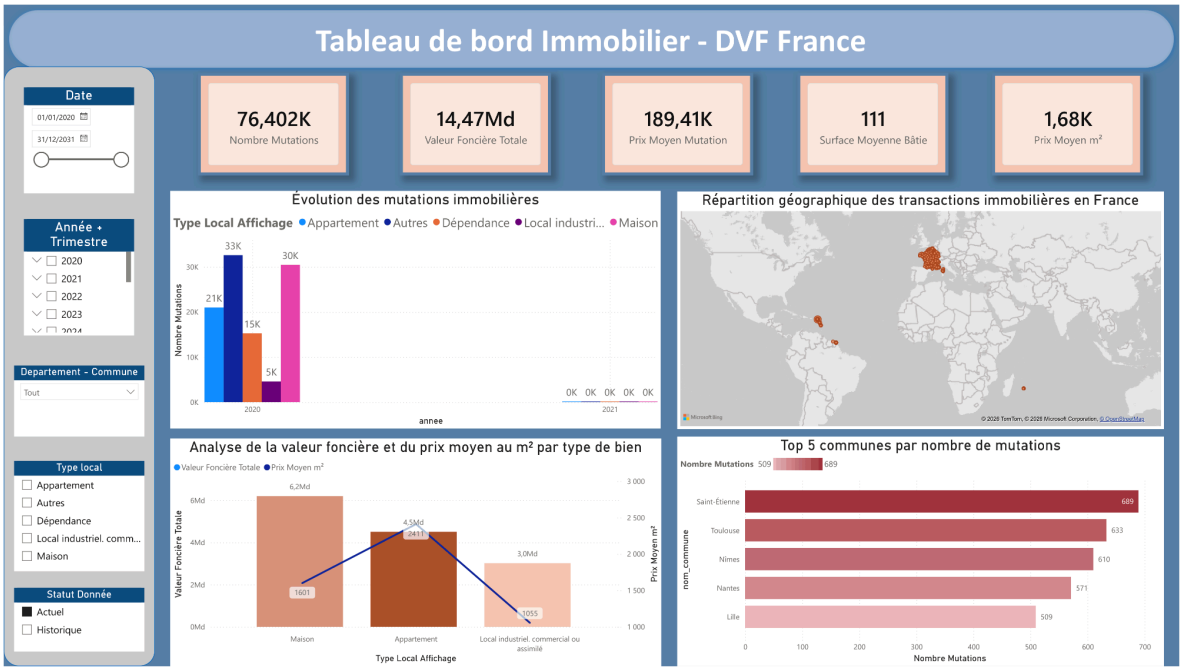


Figure 23. Tableau de bord de synthèse du marché immobilier français

La page Synthèse constitue la page d'accueil du tableau de bord et offre une vue globale du marché immobilier français à partir des données DVF intégrées dans le Data Warehouse.

Plusieurs filtres interactifs permettent de sélectionner une période d'analyse, un département, une commune, un type de local ou encore le statut des données (actuelles ou historiques).

Les indicateurs clés montrent que le jeu de données contient plus de 76 000 mutations immobilières, représentant une valeur foncière totale de 14,47 milliards d'euros. Le prix moyen par mutation est estimé à 189 410 €, tandis que la surface moyenne bâtie atteint 111 m².

L'analyse des mutations met en évidence une forte activité sur les biens résidentiels, notamment les maisons et les appartements, qui représentent la majorité des transactions observées. Les maisons concentrent la valeur foncière la plus importante, tandis que les appartements affichent le prix moyen au mètre carré le plus élevé.

La carte géographique permet d'identifier les zones les plus dynamiques du territoire. Le classement des communes selon le nombre de mutations montre que Saint-Étienne, Toulouse, Nîmes, Nantes et Lille figurent parmi les communes les plus actives du périmètre étudié.

Cette page fournit ainsi une vision synthétique des principales tendances du marché immobilier et constitue le point d'entrée des analyses réalisées dans les autres pages du tableau de bord.

B. Page Lot

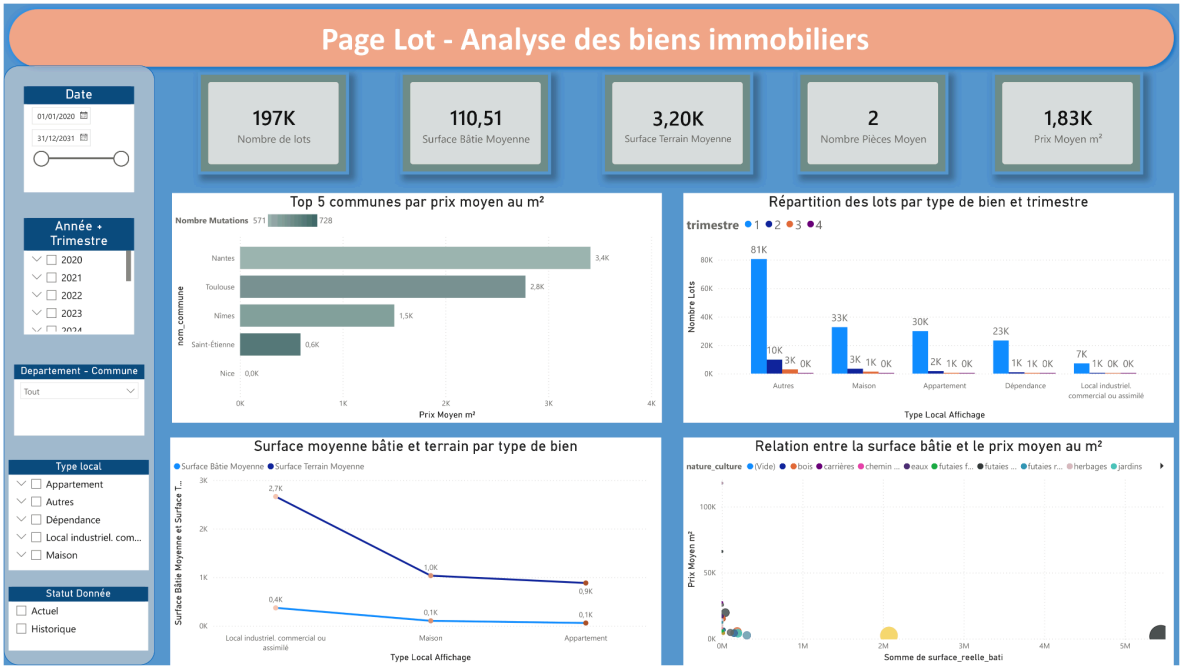


Figure 24. Page Lot – Analyse des caractéristiques des biens immobiliers

La page Lot est dédiée à l'analyse des caractéristiques physiques des biens immobiliers présents dans la base DVF. Elle permet d'étudier les surfaces, les types de biens ainsi que les prix au mètre carré selon différents critères géographiques.

Plusieurs filtres interactifs permettent de sélectionner une période d'analyse, une commune, un département, un type de local ou encore le statut des données afin d'adapter les résultats aux besoins des utilisateurs.

Les indicateurs de synthèse montrent que le Data Warehouse contient près de 197 000 lots immobiliers. La surface bâtie moyenne est d'environ 111 m², tandis que la surface moyenne des terrains atteint 3 200 m². Le prix moyen au mètre carré est estimé à 1 830 €/m².

L'analyse met en évidence des différences importantes entre les territoires et les catégories de biens Certaines communes présentent des prix moyens au mètre carré particulièrement élevés, traduisant des disparités importantes entre les territoires. Les maisons et les appartements représentent la majorité des lots enregistrés, confirmant la prédominance du marché résidentiel dans les données étudiées.

Les locaux industriels et commerciaux se distinguent par des surfaces bâties et des terrains généralement plus importants que les autres catégories de biens. À l'inverse, les appartements présentent les surfaces les plus réduites mais affichent souvent une valorisation plus élevée au mètre carré.

Cette page permet ainsi d'analyser les caractéristiques physiques des biens immobiliers et d'identifier les écarts de valorisation selon les territoires et les types de biens.

C. Page Mutation

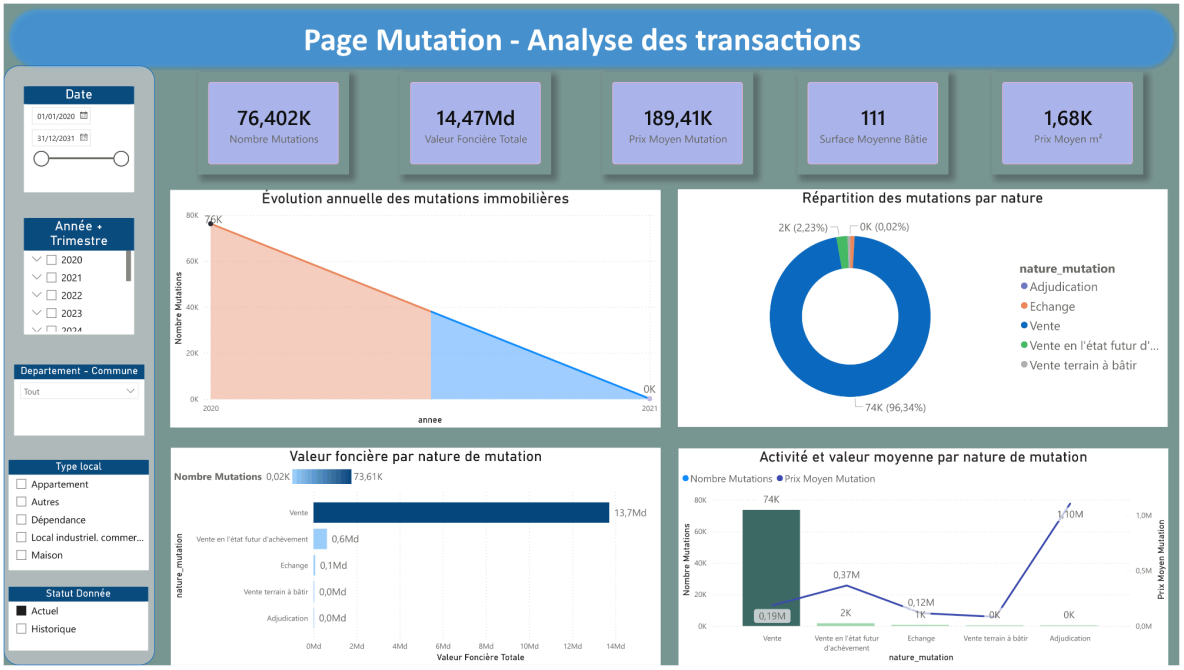


Figure 25. Page Mutation – Analyse des transactions immobilières

La page Mutation est dédiée à l'analyse des transactions immobilières enregistrées dans la base DVF. Elle permet d'étudier l'évolution des mutations dans le temps, leur répartition selon leur nature ainsi que leur contribution à la valeur foncière du marché immobilier.

Plusieurs filtres interactifs permettent de sélectionner une période d'analyse, un département, une commune, un type de local ou encore le statut des données afin d'adapter les résultats aux besoins des utilisateurs.

Les indicateurs de synthèse montrent que le périmètre étudié comprend plus de 76 000 mutations immobilières, représentant une valeur foncière totale de 14,47 milliards d'euros. Le prix moyen par mutation est estimé à 189 410 €, tandis que le prix moyen au mètre carré atteint environ 1 680 €/m².

L'analyse met en évidence une forte concentration des transactions sur l'année 2020, qui représente la majorité des mutations présentes dans le jeu de données. La répartition par nature de mutation montre également une domination très nette des ventes classiques, qui représentent plus de 96 % des transactions enregistrées.

Cette tendance se retrouve dans l'analyse de la valeur foncière, les ventes classiques générant la quasi-totalité de la valeur observée. Les autres catégories de mutations, telles que les échanges, les adjudications ou les ventes de terrains à bâtir, restent beaucoup plus marginales. Cette forte concentration confirme que les ventes classiques constituent le principal moteur du marché immobilier observé dans les données DVF.

Cette page permet ainsi d'identifier les principaux types de transactions, leur poids économique et leur contribution à la dynamique globale du marché immobilier.

D. Page Localisation

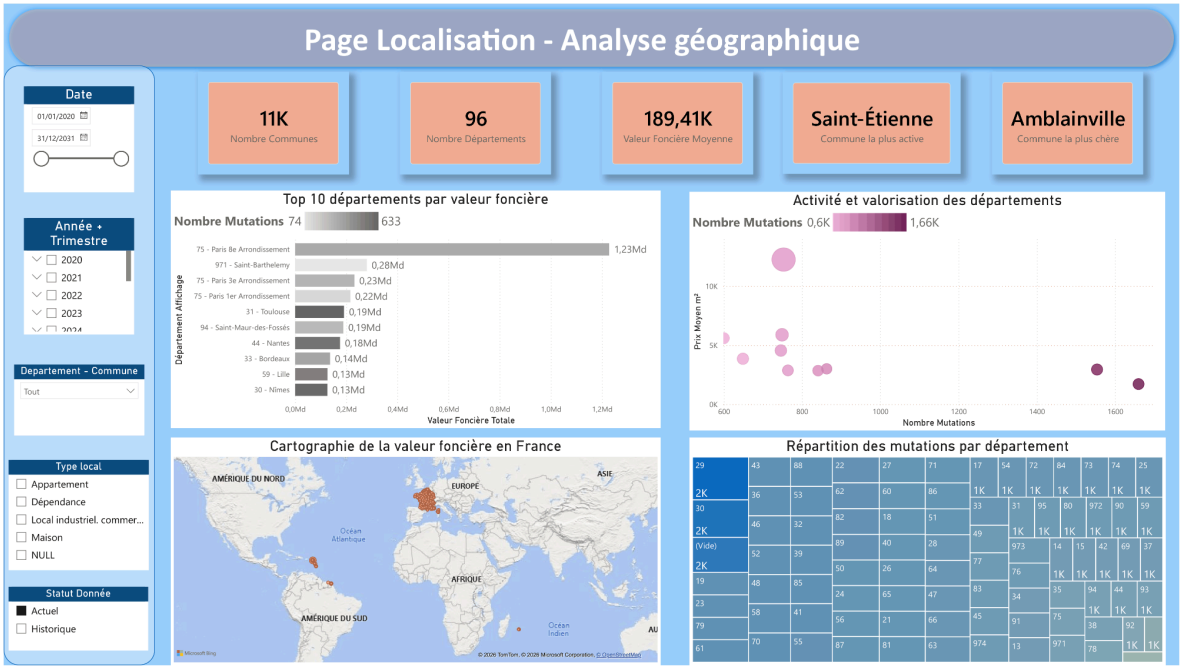


Figure 26. Page Localisation – Analyse géographique du marché immobilier

La page Localisation est dédiée à l'analyse géographique des transactions immobilières enregistrées dans la base DVF. Elle permet d'identifier les territoires les plus actifs, les plus valorisés et les disparités observées entre les différentes zones géographiques.

Plusieurs filtres interactifs permettent de sélectionner une période d'analyse, un département, une commune, un type de local ou encore le statut des données afin d'adapter les résultats aux besoins des utilisateurs.

Les indicateurs de synthèse montrent que le périmètre étudié couvre près de 11 000 communes réparties sur 96 départements. La valeur foncière moyenne est estimée à 189 410 € par mutation. Parmi les communes analysées, Saint-Étienne figure parmi les territoires les plus actifs en nombre de transactions, tandis que certaines communes présentent des niveaux de valorisation foncière particulièrement élevés.L'analyse met en évidence une forte concentration de la valeur foncière dans certains territoires, notamment en Île-de-France, avec une domination du département de Paris (75). Les départements les plus dynamiques concentrent à la fois un nombre important de mutations et des niveaux de valorisation élevés.

La cartographie permet de visualiser la répartition spatiale des transactions immobilières sur l'ensemble du territoire français et d'identifier rapidement les principales zones d'activité du marché. Les visualisations mettent également en évidence les écarts entre les départements en termes de volume de transactions et de valeur foncière.

Cette page permet ainsi d'analyser les disparités géographiques du marché immobilier français et d'identifier les territoires les plus attractifs selon différents critères d'analyse.

E. Page Parcelle

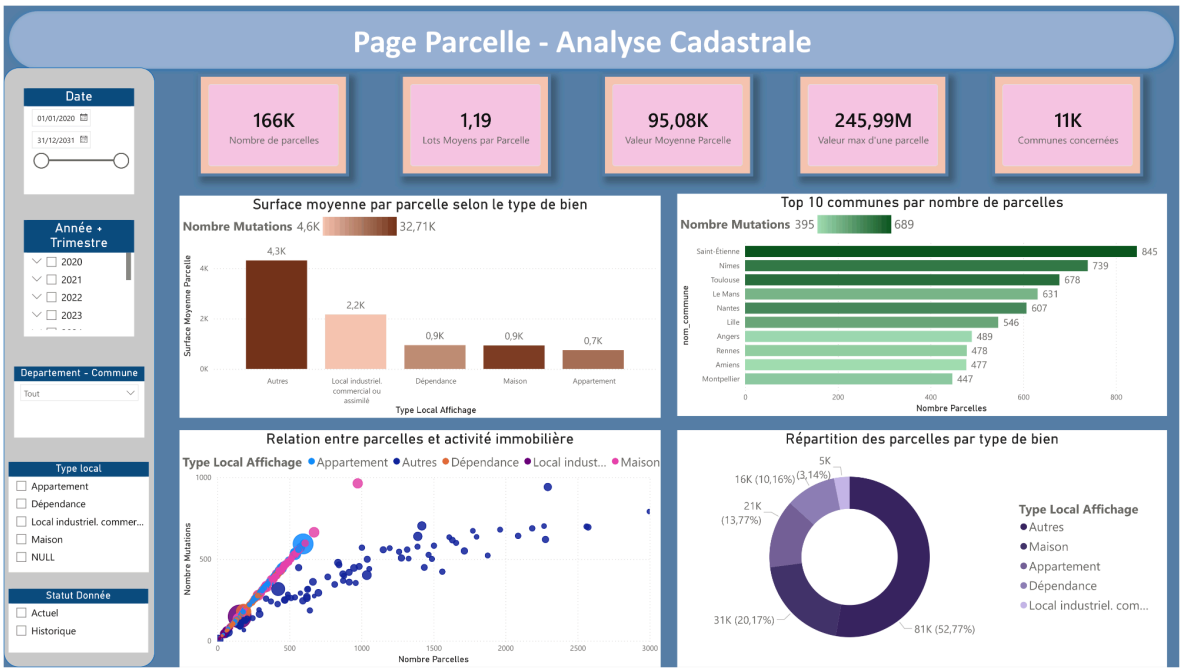


Figure 27. Page Parcelle – Analyse des caractéristiques cadastrales

La page Parcelle est dédiée à l'analyse cadastrale des biens immobiliers présents dans la base DVF. Elle permet d'étudier les caractéristiques des parcelles, leur répartition selon les types de biens ainsi que leur lien avec l'activité immobilière observée sur le territoire.

Plusieurs filtres interactifs permettent d'affiner les analyses selon la période étudiée, le département, la commune, le type de local ou encore le statut des données.

Les indicateurs de synthèse montrent que le périmètre étudié comprend près de 166 000 parcelles réparties sur environ 11 000 communes. Chaque parcelle est associée en moyenne à 1,19 lot, tandis que la valeur moyenne par parcelle est estimée à 95 080 €.

L'analyse met en évidence des différences importantes selon les types de biens. Les catégories Autres ainsi que les locaux industriels et commerciaux présentent les surfaces moyennes les plus élevées, alors que les appartements sont associés aux surfaces les plus faibles. La répartition des parcelles confirme également la prédominance des catégories résidentielles dans les données étudiées.

Le classement des communes selon le nombre de parcelles montre une forte concentration de l'activité cadastrale dans certaines villes, notamment Saint-Étienne, Nîmes et Toulouse. Par ailleurs, l'analyse de la relation entre le nombre de parcelles et le nombre de mutations met

en évidence une corrélation positive : les communes disposant d'un parc cadastral important enregistrent généralement davantage de transactions immobilières.

Cette page permet ainsi d'étudier la structure cadastrale du territoire et de mieux comprendre les relations entre les parcelles et l'activité immobilière observée dans les données DVF.

5. Perspectives

Ce projet a permis de mettre en place une chaîne décisionnelle complète, depuis l'intégration des données DVF jusqu'à leur visualisation dans Power BI. Bien que les objectifs initiaux aient été atteints, plusieurs pistes d'amélioration peuvent être envisagées afin d'enrichir davantage les analyses et d'améliorer les performances du système.

D'un point de vue technique, une première évolution consisterait à automatiser entièrement le processus d'alimentation du Data Warehouse. Actuellement, certaines étapes d'exportation et d'importation nécessitent encore des interventions manuelles. La mise en place d'un processus ETL automatisé permettrait d'actualiser les données de manière régulière tout en réduisant les risques d'erreurs.

Une autre amélioration concernerait l'optimisation des performances. Compte tenu du volume important des données DVF, l'utilisation de techniques de partitionnement, d'indexation avancée ou d'actualisation incrémentielle dans Power BI permettrait de réduire les temps de chargement et d'améliorer l'expérience utilisateur.

Sur le plan analytique, de nouveaux indicateurs pourraient être développés afin d'approfondir l'étude du marché immobilier. Il serait notamment possible d'intégrer des analyses temporelles plus détaillées, telles que l'évolution des prix immobiliers par année, par région ou par type de bien. Des indicateurs de variation et de croissance pourraient également être ajoutés afin de mieux suivre les tendances du marché.

Le tableau de bord pourrait également être enrichi par l'intégration de données externes, telles que les données socio-économiques de l'INSEE, les taux d'intérêt immobiliers, les données démographiques ou encore les informations relatives aux transports et aux infrastructures. Le croisement de ces différentes sources permettrait d'obtenir une vision plus complète des facteurs influençant le marché immobilier.

Enfin, une perspective particulièrement intéressante consisterait à développer des modèles de prédiction à l'aide de techniques de Data Science et de Machine Learning. Ces modèles pourraient être utilisés pour estimer la valeur foncière d'un bien, anticiper l'évolution des prix immobiliers ou identifier les zones présentant un fort potentiel de développement. Cette approche transformerait le système décisionnel actuel en un véritable outil d'aide à la décision prédictive.

Ainsi, cette SAE constitue une base solide pour le développement futur d'un système décisionnel enrichi, capable d'intégrer des analyses avancées, des données externes et des

modèles prédictifs afin d'améliorer la compréhension et l'anticipation des évolutions du marché immobilier.

CONCLUSION

1. Bilan de projet

Cette SAE nous a permis de mobiliser l'ensemble des compétences acquises au cours du BUT Science des Données à travers la réalisation d'un projet décisionnel complet et proche des problématiques rencontrées en entreprise. Contrairement aux travaux pratiques réalisés auparavant avec Oracle et ODI, où les étapes d'intégration étaient souvent guidées et réalisées manuellement, nous avons cette fois conçu et développé notre propre architecture de données composée des couches SOURCE, DSA, ODS et DWH. Nous avons également développé un pipeline ETL en Python permettant d'automatiser l'intégration, la transformation et l'historisation des données, puis construit un Data Warehouse en étoile destiné à l'analyse décisionnelle.

Ce projet nous a permis de mettre en pratique des compétences techniques très recherchées dans le monde professionnel, notamment en modélisation de données, en développement ETL, en gestion de bases de données et en Business Intelligence. Il nous a également permis de mieux comprendre le fonctionnement réel d'un projet Data, depuis l'exploitation de données brutes jusqu'à la production d'indicateurs d'aide à la décision.

À partir des données DVF, nous avons conçu plusieurs tableaux de bord interactifs sous Power BI afin d'analyser le marché immobilier français selon différents axes : les mutations immobilières, la localisation géographique, les parcelles cadastrales et les types de biens. Cette étude nous a également permis de mieux comprendre certaines caractéristiques du marché immobilier en France et les disparités existantes entre les territoires.

Malgré les difficultés rencontrées liées au volume important des données, à leur qualité et à la complexité des flux d'intégration entre les différentes couches du système décisionnel, nous avons atteint les objectifs fixés au début du projet. Cette expérience a renforcé nos compétences techniques, notre capacité d'analyse, notre autonomie ainsi que notre aptitude à travailler en équipe sur un projet de grande envergure, similaire à ceux que nous pourrons rencontrer dans notre future vie professionnelle.

2. Bilan personnel

Cette SAE nous a permis de développer de nombreuses compétences techniques et organisationnelles acquises au cours de notre formation en BUT Science des Données.

Tout d'abord, nous avons appris à mieux gérer notre temps et à organiser notre travail sur un projet de longue durée. La planification des différentes étapes, le respect des délais et la coordination entre les membres du binôme nous ont permis d'améliorer notre efficacité et notre autonomie.

Ce projet nous a également permis de valoriser les points forts de chacun. En répartissant les tâches selon nos compétences respectives, nous avons pu travailler de manière complémentaire tout en partageant nos connaissances. Cette collaboration nous a appris à communiquer efficacement, à prendre des décisions ensemble et à résoudre rapidement les problèmes rencontrés.

Sur le plan technique, nous avons renforcé nos compétences en modélisation de bases de données, en développement de pipelines ETL avec Python, en SQL, en Data Warehouse et en Business Intelligence avec Power BI. Nous avons également appris à manipuler des volumes importants de données, à contrôler leur qualité et à mettre en place des solutions adaptées aux contraintes rencontrées.

Enfin, cette SAE nous a permis de mieux comprendre la manière dont les données sont exploitées dans un contexte professionnel. Nous avons découvert l'ensemble du cycle de vie de la donnée, depuis son intégration jusqu'à sa valorisation à travers des tableaux de bord décisionnels. Cette expérience nous a apporté une vision plus concrète des métiers de la Data et nous a permis de développer des compétences directement applicables dans notre future vie professionnelle.

GLOSSAIRE

DVF (Demandes de Valeurs Foncières)

DGFiP (Direction Générale des Finances Publiques)

DSA (Data Staging Area)

DRY (Don’t Repeat Yourself)

DWH (Data Warehouse)

MPD (Modèle Physique de Données)

MLD (Modèle Logique de données)

MCD (Modèle Conceptuel de Données)

ODS (Operational Data Store)

ANNEXE

CSV - Source

	table_name	colonnes
0	commune	code_commune, nom_commune, ancien_code_commune...
1	local	code_type_local, type_local
2	culture	code_nature_culture, nature_culture
3	culture_speciale	code_nature_culture_speciale, nature_culture_s...
4	mutation	id_mutation, nature_mutation, date_mutation, v...
5	parcelle	id_parcelle, ancien_id_parcelle, numero_dispos...
6	adresse	id_adresse, adresse_numero, adresse_suffixe, a...
7	lot	id_lot, lot1_numero, lot1_surface_carrez, lot2...

```
Table commune chargée avec succès : 648 lignes.
Table local chargée avec succès : 4 lignes.
Table culture chargée avec succès : 19 lignes.
Table culture_speciale chargée avec succès : 27 lignes.
Table mutation chargée avec succès : 163413 lignes.
Table parcelle chargée avec succès : 197692 lignes.
Table adresse chargée avec succès : 137280 lignes.
Table lot chargée avec succès : 340454 lignes.
```

```
(648, 5)
```

	code_commune	nom_commune	ancien_code_commune	ancien_nom_commune	code_departement
0	59001	Abancourt	None	None	59
1	59002	Abscon	None	None	59
2	59003	Aibes	None	None	59
3	59004	Aix-en-Pévèle	None	None	59
4	59005	Allennes-les-Marais	None	None	59

Source – ODS

	table_name	priority	colonnes
0	commune	0	code_commune, nom_commune, ancien_code_commune...
1	local	1	code_type_local, type_local
2	culture	2	code_nature_culture, nature_culture
3	culture_speciale	3	code_nature_culture_speciale, nature_culture_s...
4	mutation	4	id_mutation, nature_mutation, date_mutation, v...
5	parcelle	5	id_parcelle, ancien_id_parcelle, numero_dispos...
6	adresse	6	id_adresse, adresse_numero, adresse_suffixe, a...
7	lot	7	id_lot, lot1_numero, lot1_surface_carrez, lot2...

(648, 5)

	code_commune	nom_commune	ancien_code_commune	ancien_nom_commune	code_departement
0	59001	Abancourt	None	None	59
1	59002	Abscon	None	None	59
2	59003	Aibes	None	None	59
3	59004	Aix-en-Pévèle	None	None	59
4	59005	Allennes-les-Marais	None	None	59

ODS – DSA

```
ods_commune est vide, insert 648 ligne(s) dans la table ods_commune
ods_local est vide, insert 4 ligne(s) dans la table ods_local
ods_culture est vide, insert 19 ligne(s) dans la table ods_culture
ods_culture_speciale est vide, insert 27 ligne(s) dans la table ods_culture_speciale
ods_mutation est vide, insert 163413 ligne(s) dans la table ods_mutation
ods_parcelle est vide, insert 197692 ligne(s) dans la table ods_parcelle
ods_adresse est vide, insert 137280 ligne(s) dans la table ods_adresse
ods_lot est vide, insert 340454 ligne(s) dans la table ods_lot
```

(648, 6)

	code_commune	nom_commune	ancien_code_commune	ancien_nom_commune	code_departement	load_date
0	59001	Abancourt	None	None	59	2026-06-04 09:08:32
1	59002	Abscon	None	None	59	2026-06-04 09:08:32
2	59003	Aibes	None	None	59	2026-06-04 09:08:32
3	59004	Aix-en-Pévèle	None	None	59	2026-06-04 09:08:32
4	59005	Allennes-les-Marais	None	None	59	2026-06-04 09:08:32

DSA – DWH

```
  dim_date est vide, insert 4383 lignes dans la table
  dim_localisation est vide, insert 137280 ligne(s)
  dim_lot est vide, insert 340454 ligne(s)
  dim_mutation est vide, insert 163413 ligne(s)
  dim_parcelle est vide, insert 197692 ligne(s)

Fait_DVF est vide, insert 340454 ligne(s)
(4383, 6)
```

	date_id	full_date	jour	mois	trimestre	annee
0	1	2020-01-01	1	1	1	2020
1	2	2020-01-02	2	1	1	2020
2	3	2020-01-03	3	1	1	2020
3	4	2020-01-04	4	1	1	2020
4	5	2020-01-05	5	1	1	2020